Creacion de Red Neuronal Fuego Net

Manual paso a paso para construir y entrenar FuegoNet,
un detector de fuego y humo para el Tarot 650

PyTorch · YOLO26 · ROS 2 Jazzy · Gazebo Harmonic

Dr. Jairo Olguín Roque
FES Acatlán, UNAM

Octubre de 2026

Índice general

Cómo usar este manual IV

I Entender lo que vamos a construir 1

1. Qué es YOLO26 y qué vamos a copiar 2

1.1. El problema: encontrar fuego y humo en una imagen . . . 2
1.2. Detectores de una etapa: la idea de YOLO . . . 2
1.3. Lo nuevo de YOLO26 . . . 3
1.4. El plan: FuegoNet . . . 3

2. Preparar el entorno y los datos 5

II Construir la red pieza por pieza 8

3. Los bloques de construcción 9

3.1. Cómo se escribe una capa en PyTorch . . . 9
3.2. Conv: la unidad mínima . . . 9
3.3. Cuello (bottleneck) y C3k . . . 10
3.4. C3k2: el bloque principal . . . 10
3.5. SPPF: ver a varias escalas . . . 11
3.6. Atención: que cada celda mire a todas las demás . . . 11

4. Ensamblar la red: columna, cuello y cabeza 13

4.1. La forma general . . . 13
4.2. Escalas: n, s y m . . . 14
4.3. La columna vertebral y el cuello . . . 14
4.4. La cabeza: de mapas a cajas . . . 16

4.5. ¿Por qué ya no hace falta NMS? 16

III Enseñarle a la red 18

5. Quién responde por cada objeto: TAL y STAL 19

5.1. El problema 19
5.2. TAL: asignación alineada a la tarea 19
5.3. Uno a muchos y uno a uno 20
5.4. STAL: que ningún foco se quede sin responsable 20
5.5. El código 21

6. La función de pérdida 22

6.1. Tres términos por cabeza 22
6.1.1. Clase: entropía cruzada binaria 22
6.1.2. Caja: CIoU 22
6.1.3. Distancias: L1 (en lugar de DFL) 23
6.2. Dos cabezas y ProgLoss 23
6.3. Detalles que importan 24

7. Leer los datos y aumentarlos 25

7.1. De archivo a tensor 25
7.2. Letterbox: cambiar de tamaño sin deformar 25
7.3. Aumentos de datos 25

8. Entrenar 27

8.1. El optimizador MuSGD 27
8.2. Tasa de aprendizaje, calentamiento y lote nominal 28
8.3. EMA: el modelo suavizado 28
8.4. El ciclo de entrenamiento 28
8.5. Qué pasó en el entrenamiento 30

IV Medir y usar la red 31

9. Medir y comparar con YOLO26n 32

9.1. mAP en pocas palabras 32

9.2. Las tres redes que se comparan . . . 33
9.3. Resultados . . . 33

10. Usar la red: imágenes, ONNX y ROS 2 36
10.1. Detectar en una imagen . . . 36
10.2. Exportar a ONNX . . . 37
10.3. Conectar FuegoNet a la simulación . . . 37

11. Qué sigue 39
11.1. Lo que ya tienes . . . 39
11.2. Cómo reportarlo . . . 39
11.3. Experimentos sugeridos . . . 39

A. Código completo 41
A.1. `bloques.py` . . . 41
A.2. `modelo.py` . . . 44
A.3. `asignador.py` . . . 47
A.4. `perdida_iou.py` . . . 49
A.5. `perdida.py` . . . 49
A.6. `datos.py` . . . 51
A.7. `optimizador.py` . . . 55
A.8. `entrenar.py` . . . 56
A.9. `metricas.py` . . . 60
A.10. `evaluar.py` . . . 61
A.11. `inferir.py` . . . 63
A.12. `exportar.py` . . . 65

Cómo usar este manual

Este manual explica, paso a paso, cómo se construyó **FuegoNet**: una red neuronal de detección de objetos con la misma arquitectura que YOLO26, escrita desde cero en PyTorch, para reconocer **fuego** y **humo** en las imágenes de la cámara del Tarot 650 durante la búsqueda de incendios forestales.

Hasta ahora la simulación usaba YOLO26n de Ultralytics: se descargan pesos entrenados en COCO (80 clases de objetos cotidianos) y se ajustan a nuestras dos clases. Funciona muy bien, pero la red es una caja negra: no se ve qué hace por dentro, no se puede modificar con libertad y es difícil justificar cada decisión. Aquí abrimos esa caja. Cada pieza de YOLO26 se programa a mano, se prueba por separado y se explica con sus ecuaciones. Al final se entrena la red completa *sin pesos preentrenados* y se compara con YOLO26n usando el mismo código de medición.

Qué necesitas saber antes

- Python básico (funciones, clases, listas) y saber correr un comando en la terminal.
- La idea de neurona, convolución, IoU y mAP. Si no la tienes, lee primero el capítulo «Redes neuronales y YOLO sin complicaciones» del libro de la simulación (`doc/manual`). Aquí se repasa lo indispensable.
- No hace falta saber PyTorch: cada vez que aparece algo nuevo se explica.

Cómo está organizado

Parte I Qué es YOLO26, qué tiene de nuevo y cómo preparar el entorno y los datos.

Parte II Los bloques de la red (convoluciones, C3k2, SPPF, atención) y cómo se ensamblan en columna vertebral, cuello y cabeza.

Parte III Cómo aprende: la asignación de etiquetas (TAL y STAL), la función de pérdida de dos cabezas, los aumentos de datos y el ciclo de entrenamiento con el optimizador MuSGD.

Parte IV Cómo se mide, cómo se compara con YOLO26n y cómo se conecta a ROS 2 para que el dron la use en vuelo.

Cada capítulo avanza en **pasos numerados**. Al final de cada paso hay un recuadro «Lo que debes ver» con la salida esperada; si lo que ves es distinto, no sigas: algo quedó mal en ese paso.

Convenciones

- Los comandos se escriben así y se corren desde `~/ros2_ws/src/cuadrirotor_sim`:

```
~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.modelo
```

- Las rutas, archivos y nombres de variables van en `monoespaciado`.
- El código completo está en `cuadrirotor_sim/red_fuego/` y en el apéndice A. En el texto solo aparecen las partes importantes.
- Los nombres del código están en español (`Cuello`, `Atencion`, `AsignadorTAL`) para que se lean junto con el texto. Entre paréntesis se da el nombre en inglés que vas a encontrar en los artículos.

Nota

Todos los números de este manual (parámetros, tiempos, mAP) salieron de correr el código en la computadora del laboratorio: Ubuntu, PyTorch 2.14 con CUDA 13 y una tarjeta NVIDIA RTX 5060 Ti de 16 GB. En otra máquina los tiempos cambian, pero las métricas deben salir parecidas.

Parte I

Entender lo que vamos a construir

Capítulo 1

Qué es YOLO26 y qué vamos a copiar

En este capítulo

Qué hace un detector de una etapa, de dónde viene la familia YOLO y cuáles son las cinco ideas de YOLO26 que vamos a programar: cabeza sin DFL, inferencia sin NMS, ProgLoss, STAL y el optimizador MuSGD.

1.1. El problema: encontrar fuego y humo en una imagen

La cámara del Tarot 650 entrega imágenes de 640×480 píxeles. Un **detector de objetos** debe responder, para cada imagen, una lista de *cajas*: dónde está cada objeto (un rectángulo x_1, y_1, x_2, y_2), qué es (fuego o humo) y qué tan seguro está (una confianza entre 0 y 1). La misión de búsqueda (`busqueda_incendio`) usa esas cajas para decidir hacia dónde volar y cuándo confirmar el incendio.

Ejemplo de la vida diaria

Piensa en un vigilante en una torre forestal. No le basta saber que «en algún lado hay humo»: necesita señalar con el dedo *dónde*, y distinguir una columna de humo de una nube baja. Eso es la detección: clasificar *y* localizar.

1.2. Detectores de una etapa: la idea de YOLO

YOLO (*You Only Look Once*, 2016) propuso mirar la imagen una sola vez. La red reduce la imagen a una **rejilla** de celdas y cada celda predice, en una sola pasada, si hay un objeto cerca de su centro, de qué clase es y dónde están los bordes de su caja. No hay una etapa previa que proponga regiones (como en Faster R-CNN), por eso es rápido y sirve en un dron.

Desde entonces la familia ha cambiado mucho. Para este manual importan tres pasos:

YOLOv8 (2023) Sin anclas predefinidas: cada celda predice distancias a los cuatro lados de la caja. Asignación de etiquetas *alineada a la tarea* (TAL). Cada distancia se predice como una distribución sobre 16 valores (DFL, *Distribution Focal Loss*).

YOLOv10 (2024) Dos cabezas: una «uno a muchos» para entrenar y otra «uno a uno» para inferir.

La segunda da una sola caja por objeto y elimina el NMS.

YOLO11 (2024) Bloques C3k2 y atención espacial C2PSA en la columna vertebral.

1.3. Lo nuevo de YOLO26

YOLO26 (Ultralytics, 2025–2026) junta lo anterior y lo simplifica pensando en equipos pequeños, como la computadora a bordo de un dron:

1. **Sin DFL** (`reg_max=1`). Cada distancia es un solo número, no una distribución de 16. La cabeza es más pequeña, la exportación a ONNX o TensorRT es directa y la pérdida usa L1 en lugar de DFL.
2. **Sin NMS, de extremo a extremo.** Usa la doble cabeza de YOLOv10. La salida ya trae una caja por objeto: basta ordenar por confianza.
3. **ProgLoss.** El peso entre las dos cabezas cambia durante el entrenamiento: empieza dominando la de muchos positivos (aprende rápido) y termina dominando la de uno a uno (la que se usa al final).
4. **STAL** (*Small-Target-Aware Label assignment*). Las cajas muy pequeñas se agrandan al elegir celdas candidatas para que nunca se queden sin «responsable». A gran altura, un foco de fuego mide pocos píxeles: esto nos importa mucho.
5. **MuSGD.** Un optimizador que mezcla SGD con Muon, un método nacido para entrenar modelos de lenguaje que «ortogonaliza» la actualización de cada matriz de pesos.

Nota

Estas cinco ideas se tomaron leyendo el código fuente de Ultralytics 8.4 (archivos `cfg/models/26/yolo26.yaml`, `nn/modules/head.py`, `utils/tal.py`, `utils/loss.py` y `optim/muon.py`). FuegoNet reproduce la arquitectura y la receta de entrenamiento, pero su código es independiente: no importa nada de Ultralytics.

1.4. El plan: FuegoNet

FuegoNet copia la escala **n** (*nano*) de YOLO26, la más pequeña y la que ya usa la simulación. La tabla 1.1 resume qué se copia y dónde está cada pieza.

Tabla 1.1: Piezas de YOLO26 y su archivo en `cuadrirotor_sim/red_fuego/`.

Pieza	**Qué hace**	**Archivo**
Conv, C3k2, SPPF, C2PSA	Bloques que extraen características	`bloques.py`
Columna, cuello y cabeza	La red completa, de la imagen a las cajas	`modelo.py`
TAL + STAL	Decide qué celdas responden por cada objeto	`asignador.py`
Pérdida E2E + ProgLoss	Mide el error de las dos cabezas	`perdida.py`
CIoU	Traslape de cajas para la pérdida	`perdida_iou.py`
Mosaico, afín, HSV	Aumentos de datos	`datos.py`
MuSGD	Optimizador	`optimizador.py`
Ciclo de entrenamiento, EMA	Entrenar y guardar el mejor modelo	`entrenar.py`
mAP50, mAP50-95	Medir	`metricas.py`, `evaluar.py`
Detector, ONNX	Usar la red	`inferir.py`, `exportar.py`

Recuerda

Que la arquitectura sea la misma no significa que los resultados lo sean. YOLO26n de la simulación partió de pesos entrenados con 118 000 imágenes de COCO; FuegoNet empieza con pesos al azar y solo ve nuestras 5 400 imágenes. Por eso, en el capítulo 9 se compara además con un YOLO26n entrenado *también desde cero*: esa es la comparación justa que valida nuestro código.

Capítulo 2

Preparar el entorno y los datos

En este capítulo

Comprobar que PyTorch ve la tarjeta gráfica, conocer el dataset de fuego y humo que genera el simulador y crear la carpeta donde vivirá la red.

Paso 1 **Comprobar el entorno de Python.** La simulación ya tiene un entorno virtual para YOLO en `~/.venvs/yolo`, con PyTorch, OpenCV y Ultralytics. FuegoNet solo necesita PyTorch, OpenCV, NumPy y PyYAML; Ultralytics se usa únicamente para comparar.

```
~/.venvs/yolo/bin/python -c "import torch, cv2, numpy; \
  print(torch.__version__, torch.cuda.is_available(), torch.cuda.get_device_name(0)); \
  print(cv2.__version__, numpy.__version__)"
```

Lo que debes ver

```
2.14.1+cu130 True NVIDIA GeForce RTX 5060 Ti
4.11.0 1.26.4
```

Cuidado

NumPy debe ser de la serie 1.x en este entorno: el puente de ROS 2 Jazzy (`cv_bridge`, `rclpy`) está compilado contra NumPy 1. Si instalas algo con `pip` y te sube NumPy a 2.x, regrésalo de inmediato:

```
~/.venvs/yolo/bin/pip install "numpy==1.26.4" "ml-dtypes<0.6"
```

Nos pasó al instalar `onnx`: pip subió NumPy a 2.5 sin avisar (capítulo 10).

Paso 2 **Conocer el dataset.** Las imágenes salen del propio simulador. `scripts/capturar_dataset.py` vuela el dron sobre los mundos `bosque` y `bosque_grande` con focos de fuego en lugares al azar y escribe, para cada foto, un archivo de texto con las cajas. Son dos datasets que se usan juntos:

Tabla 2.1: Dataset de fuego y humo (imágenes de 640 × 480).

Dataset	Parte	Imágenes	Cajas fuego	Cajas humo	Sin objetos
`incendio_bosque`	train	2 700	1 927	2 203	309
	val	300	213	239	40
`incendio_bosque_grande`	train	2 700	891	1 898	755
	val	300	98	213	81
Total	train	5 400	2 818	4 101	1 064
	val	600	311	452	121

Las imágenes «sin objetos» son importantes: le enseñan a la red que un claro iluminado o una roca oscura *no* son fuego (una roca dio falsos positivos al principio; por eso se agregó el mundo grande).

El formato es el de YOLO. Cada imagen `images/train/00001.jpg` tiene su `labels/train/00001.txt` con una línea por objeto:

```
0 0.472461 0.706724 0.070710 0.106154
1 0.480463 0.596074 0.145035 0.212378
```

El primer número es la clase (0 = fuego, 1 = humo) y los otros cuatro son el centro (c_x, c_y), el ancho w y el alto h de la caja, *divididos* entre el ancho y el alto de la imagen (todo queda entre 0 y 1). Un archivo vacío significa «no hay nada que detectar».

Los dos datasets se juntan con un `data.yaml` que lista ambas carpetas:

```
train:
  - /home/jairo/datasets/incendio_bosque/images/train
  - /home/jairo/datasets/incendio_bosque_grande/images/train
val:
  - /home/jairo/datasets/incendio_bosque/images/val
  - /home/jairo/datasets/incendio_bosque_grande/images/val
names:
  0: fuego
  1: humo
```

Está en `~/datasets/combinado_incendio_bosque_incendio_bosque_grande/` (lo creó `scripts/entrenar_yolo.py`).

Para curiosos (te lo puedes saltar)

De las 6 919 cajas de entrenamiento, 195 tienen un lado menor de 16 píxeles. Son focos vistos desde lejos. Sin un cuidado especial (STAL, capítulo 5) esas cajas pueden quedar sin ninguna celda que aprenda de ellas.

Paso 3 **Crear la carpeta de la red.** FuegoNet vive dentro del paquete de ROS 2 como un subpaquete de Python, para que el nodo detector la pueda importar después:

```
cd ~/ros2_ws/src/cuadrirotor_sim
mkdir -p cuadrirotor_sim/red_fuego
touch cuadrirotor_sim/red_fuego/__init__.py
```

Al terminar el manual la carpeta tendrá estos archivos:

```
cuadrirotor_sim/red_fuego/
```

```
__init__.py        bloques.py      modelo.py      asignador.py
perdida_iou.py     perdida.py      datos.py       optimizador.py
metricas.py        entrenar.py     evaluar.py     inferir.py
exportar.py
```

Como son módulos de un paquete, se ejecutan con `python-m` desde la carpeta del paquete (así funcionan los `import` relativos como `from.bloquesimportConv`).

Parte II

Construir la red pieza por pieza

Capítulo 3

Los bloques de construcción

En este capítulo

Las seis piezas con las que se arma YOLO26, cada una como una clase de PyTorch: `Conv`, `Cuello`, `C3k2`, `SPPF`, `Atencion` y `C2PSA`. Todo está en `red_fuego/bloques.py`.

3.1. Cómo se escribe una capa en PyTorch

En PyTorch toda capa es una clase que hereda de `nn.Module`. En `__init__` se crean las sub-capas (las que tienen pesos) y en `forward` se dice qué se hace con la entrada. Los datos viajan como **tensores** de cuatro dimensiones (B, C, H, W): B imágenes del lote, C canales y $H \times W$ píxeles. Una imagen RGB de 640×640 entra como $(1, 3, 640, 640)$.

3.2. Conv: la unidad mínima

Paso 4 **Escribir `Conv`.** Toda convolución de YOLO va seguida de una normalización por lotes (BatchNorm) y de la activación SiLU, $\mathrm{SiLU}(x) = x\,\sigma(x)$:

```
class Conv(nn.Module):
    def __init__(self, c1, c2, k=1, s=1, g=1, d=1, act=True):
        super().__init__()
        self.conv = nn.Conv2d(c1, c2, k, s, relleno(k, d), groups=g, dilation=d, bias=False)
        self.bn = nn.BatchNorm2d(c2, eps=1e-3, momentum=0.03)
        self.act = nn.SiLU(inplace=True) if act else nn.Identity()

    def forward(self, x):
        return self.act(self.bn(self.conv(x)))
```

- `c1`, `c2`: canales de entrada y de salida. `k`: tamaño del filtro.
- `s`: paso (*stride*). Con `s=2` el mapa sale a la mitad de alto y de ancho: así es como la red «se aleja» de la imagen.
- `relleno(k)` $= \lfloor k/2 \rfloor$ conserva el tamaño cuando `s=1`.

- No lleva sesgo (`bias=False`) porque BatchNorm ya suma uno.
- `g` (grupos) permite la convolución *en profundidad* (`DWConv`): cada canal tiene su propio filtro y no se mezcla con los demás. Cuesta C veces menos y la usa la cabeza de clases.

Una `Conv` 3×3 de 64 a 128 canales tiene $64\cdot128\cdot9 = 73\,728$ pesos más $2\cdot128$ de BatchNorm: 73 984 parámetros.

3.3. Cuello (bottleneck) y C3k

Paso 5 **Escribir el cuello de botella residual.** Dos `Conv` 3×3 y un atajo que suma la entrada: $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$. El atajo deja pasar el gradiente sin obstáculos y permite apilar muchas capas (la idea de ResNet).

```
class Cuello(nn.Module):
    def __init__(self, c1, c2, atajo=True, k=(3, 3), e=0.5):
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = Conv(c1, c_, k[0])
        self.cv2 = Conv(c_, c2, k[1])
        self.suma = atajo and c1 == c2

    def forward(self, x):
        y = self.cv2(self.cv1(x))
        return x + y if self.suma else y
```

`C3k` es un CSP (*Cross Stage Partial*) de tres convoluciones: la entrada se proyecta por dos caminos; uno pasa por dos cuellos y el otro no; al final se concatenan. Es más rico que un cuello solo y lo usan las etapas profundas.

3.4. C3k2: el bloque principal

Paso 6 **Escribir `C3k2`.** Es el bloque que más se repite en YOLO26 (8 de las 23 capas). La figura 3.1 muestra su flujo:

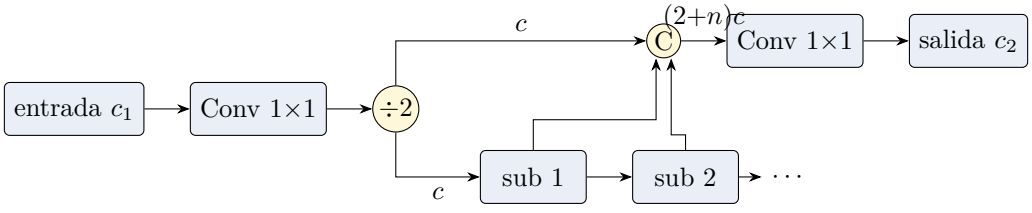


Figura 3.1: C3k2. La entrada se proyecta a $2c$ canales y se parte en dos. La segunda mitad pasa por n subbloques en cadena; *todas* las salidas intermedias se concatenan (C) y una Conv 1×1 las mezcla. El subbloque es un `Cuello`, un `C3k` o un `Cuello` + atención.

```
class C3k2(nn.Module):
    def __init__(self, c1, c2, n=1, c3k=False, e=0.5, atencion=False, atajo=True):
        super().__init__()
```

```
        self.c = int(c2 * e)
        self.cv1 = Conv(c1, 2 * self.c, 1)
        self.cv2 = Conv((2 + n) * self.c, c2, 1)
        if atencion:
            sub = lambda: nn.Sequential(Cuello(self.c, self.c, atajo),
                                        BloquePSA(self.c, cabezas=max(self.c // 64, 1)))
        elif c3k:
            sub = lambda: C3k(self.c, self.c, 2, atajo)
        else:
            sub = lambda: Cuello(self.c, self.c, atajo)
        self.m = nn.ModuleList(sub() for _ in range(n))

    def forward(self, x):
        y = list(self.cv1(x).chunk(2, 1))      # dos mitades de c canales
        for m in self.m:
            y.append(m(y[-1]))                  # cada subbloque toma la última salida
        return self.cv2(torch.cat(y, 1))
```

Consejo

El parámetro `e` (expansión) controla el ancho interno. YOLO26 usa `e=0.25` en las dos primeras etapas (mapas grandes, conviene ahorrar cómputo) y `e=0.5` en las demás.

3.5. SPPF: ver a varias escalas

Paso 7 **Escribir `SPPF`.** Al final de la columna vertebral el mapa mide 20×20. Para que cada celda vea su contexto (¿esa mancha naranja está junto a una columna de humo?), `SPPF` aplica tres *max-pool* de 5×5 en cadena. Tres ventanas de 5 equivalen a ventanas de 5, 9 y 13 celdas, sin agregar ningún parámetro. Las cuatro versiones se concatenan y se mezclan; YOLO26 agrega además un atajo $\mathbf{y} \leftarrow \mathbf{y} + \mathbf{x}$.

```
    def forward(self, x):
        y = [self.cv1(x)]
        for _ in range(self.n):              # n = 3
            y.append(self.m(y[-1]))          # MaxPool2d(5, 1, 2)
        y = self.cv2(torch.cat(y, 1))
        return x + y if self.suma else y
```

3.6. Atención: que cada celda mire a todas las demás

Paso 8 **Escribir `Atencion`, `BloquePSA` y `C2PSA`.** Una convolución solo mira a sus vecinos. La **autoatención** deja que cada celda consulte a *todas* las demás. Si el mapa tiene $N = H\,W$ celdas, de cada una se calculan tres vectores con convoluciones 1×1: una *consulta* $\mathbf{q}_i$, una *llave* $\mathbf{k}_j$ y un *valor* $\mathbf{v}_j$. La salida de la celda i es un promedio pesado de los valores:

$$\mathbf{y}_i = \sum_{j=1}^{N} a_{ij}\,\mathbf{v}_j, \qquad a_{ij} = \frac{\exp(\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_k})}{\sum_l \exp(\mathbf{q}_i \cdot \mathbf{k}_l / \sqrt{d_k})}. \tag{3.1}$$

El peso a_{ij} es grande cuando la consulta de i «se parece» a la llave de j. Con varias *cabezas* se hacen varias atenciones en paralelo, cada una con su parte de los canales. Se suma además una convolución 3×3 en profundidad sobre $\mathbf{v}$, que le dice a la red dónde está cada celda (codificación posicional).

```
    def forward(self, x):
        B, C, H, W = x.shape
        q, k, v = self.qkv(x).view(B, self.cabezas, 2 * self.dk + self.dc, H * W).split(
            [self.dk, self.dk, self.dc], dim=2)
        a = ((q * self.escala).transpose(-2, -1) @ k).softmax(dim=-1)   # (B, h, N, N)
        y = (v @ a.transpose(-2, -1)).view(B, C, H, W) + self.pe(v.reshape(B, C, H, W))
        return self.proy(y)
```

Cuidado

La matriz a tiene $N \times N$ elementos. Con el mapa de 20×20 son $400^2 = 160\,000$ por cabeza: barato. Con el de 80×80 serían 41 millones. Por eso YOLO26 solo pone atención en la etapa más profunda (P_5).

`BloquePSA` es como un bloque de transformador: atención con atajo y luego una pequeña red de avance (dos Conv 1×1) también con atajo. `C2PSA` aplica la atención solo a la mitad de los canales (estilo CSP) y la otra mitad pasa directo.

Paso 9 **Probar cada bloque.** Antes de ensamblar nada hay que comprobar que cada bloque recibe y entrega las formas correctas:

```
import torch
from cuadrirotor_sim.red_fuego.bloques import Conv, Cuello, C3k2, SPPF, C2PSA
x = torch.randn(1, 64, 80, 80)
for nombre, b in [('Conv 3x3 s=2', Conv(64, 128, 3, 2)), ('Cuello', Cuello(64, 64)),
                  ('C3k2 n=1', C3k2(64, 128, 1, False, 0.25)), ('C3k2 c3k', C3k2(64, 128, 1, True))
    ,
                  ('SPPF', SPPF(64, 64, atajo=True)), ('C2PSA', C2PSA(64, 64))]:
    y = b(x)
    print(nombre, tuple(x.shape), '->', tuple(y.shape), sum(p.numel() for p in b.parameters()))
```

Lo que debes ver

```
Conv 3x3 s=2   (1, 64, 80, 80) -> (1, 128, 40, 40)   73984 parámetros
Cuello         (1, 64, 80, 80) -> (1, 64, 80, 80)   37056 parámetros
C3k2 n=1       (1, 64, 80, 80) -> (1, 128, 80, 80)   26080 parámetros
C3k2 c3k       (1, 64, 80, 80) -> (1, 128, 80, 80)   78848 parámetros
SPPF           (1, 64, 80, 80) -> (1, 64, 80, 80)   10432 parámetros
C2PSA          (1, 64, 80, 80) -> (1, 64, 80, 80)   16352 parámetros
```

Pruébalo tú

Cambia `e=0.25` por `e=0.5` en el `C3k2n=1` y vuelve a contar los parámetros. ¿Por qué crecen casi cuatro veces si solo duplicaste el ancho? (Pista: en una convolución los pesos son $c_{\text{in}} \cdot c_{\text{out}} \cdot k^2$.)

Capítulo 4

Ensamblar la red: columna, cuello y cabeza

En este capítulo

Cómo se apilan los bloques para formar FuegoNet-n, cómo la cabeza convierte mapas de características en cajas y por qué la salida ya no necesita NMS. Todo está en `red_fuego/modelo.py`.

4.1. La forma general

Un detector moderno tiene tres partes (figura 4.1):

Columna vertebral (*backbone*) reduce la imagen paso a paso (a 1/2, 1/4, 1/8, 1/16 y 1/32) y aumenta los canales. Los mapas pequeños «saben» mucho (contexto) pero ven con poco detalle.

Cuello (*neck*, PAN-FPN) mezcla las escalas: primero de arriba abajo (el contexto de P_5 baja a P_4 y P_3) y luego de abajo arriba (el detalle de P_3 sube a P_4 y P_5).

Cabeza (*head*) en cada celda de las tres escalas predice 4 distancias y 2 puntuaciones de clase.

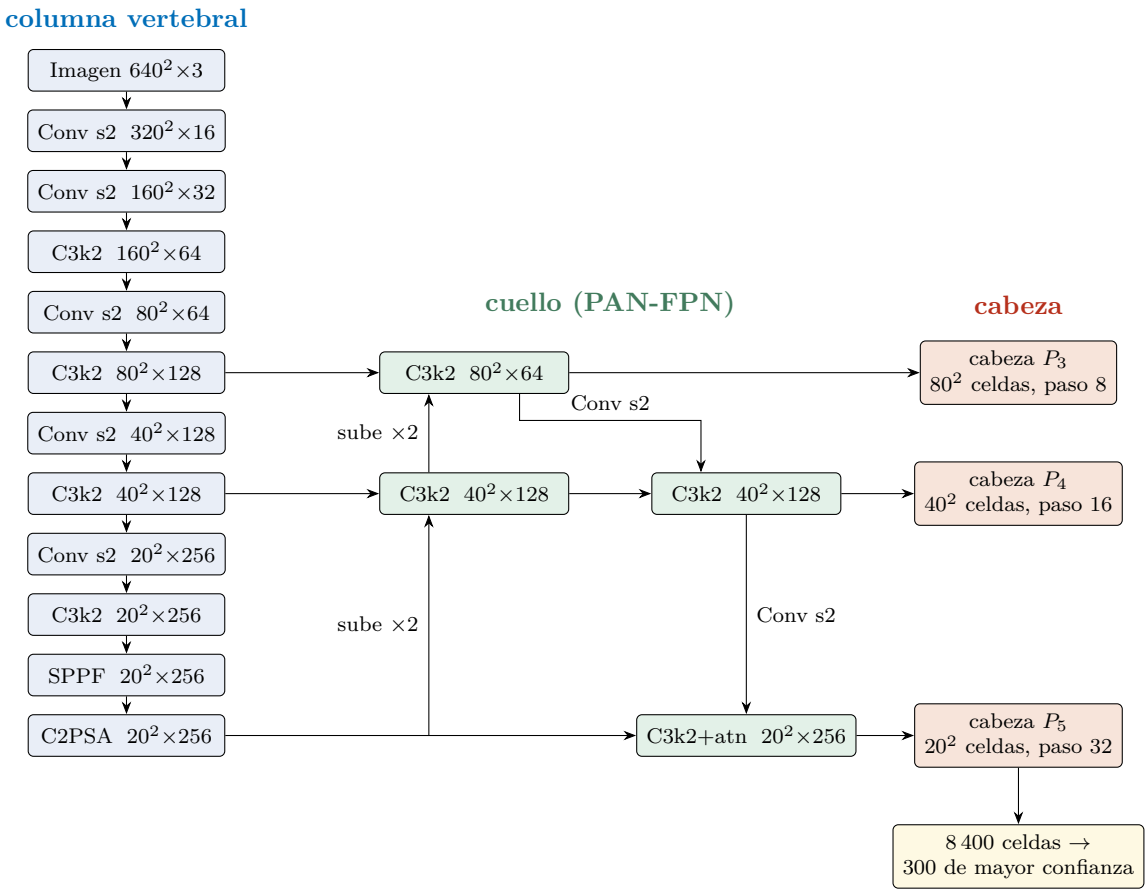


Figura 4.1: FuegoNet-n con una imagen de 640 × 640. Cada caja muestra el tamaño del mapa (alto×ancho×canales). Las flechas al cuello son concatenaciones.

4.2. Escalas: n, s y m

YOLO26 define cada tamaño con dos números: un factor de profundidad (cuántas veces se repite cada bloque) y uno de anchura (cuántos canales). El archivo `yolo26.yaml` escribe la red con los canales de la versión grande y se multiplican:

```
ESCALAS = {'n': (0.50, 0.25, 1024), 's': (0.50, 0.50, 1024), 'm': (0.50, 1.00, 512)}
...
c = lambda x: int(math.ceil(min(x, cmax) * anch / 8) * 8)   # canales múltiplos de 8
n = lambda x: max(round(x * prof), 1)                        # repeticiones
```

Así, la capa que en el *yaml* tiene 1 024 canales queda con $1024 \cdot 0.25 = 256$ en la versión **n**.

4.3. La columna vertebral y el cuello

Paso 10 **Escribir `FuegoNet.__init__`.** Se copian las 23 capas de `yolo26.yaml` en el mismo orden. Los nombres `b0`–`b10` y `h13`–`h22` conservan el número de capa del *yaml* para poder compararlos:

```
        self.b0 = Conv(3, c(64), 3, 2)                                  # P1/2
```

```
        self.b1 = Conv(c(64), c(128), 3, 2)                                  # P2/4
        self.b2 = C3k2(c(128), c(256), n(2), c3k, 0.25)
        self.b3 = Conv(c(256), c(256), 3, 2)                                 # P3/8
        self.b4 = C3k2(c(256), c(512), n(2), c3k, 0.25)                      # -> P3
        self.b5 = Conv(c(512), c(512), 3, 2)                                 # P4/16
        self.b6 = C3k2(c(512), c(512), n(2), True)                           # -> P4
        self.b7 = Conv(c(512), c(1024), 3, 2)                                # P5/32
        self.b8 = C3k2(c(1024), c(1024), n(2), True)
        self.b9 = SPPF(c(1024), c(1024), 5, 3, atajo=True)
        self.b10 = C2PSA(c(1024), c(1024), n(2))                             # -> P5
        self.sube = nn.Upsample(None, 2, 'nearest')
        self.h13 = C3k2(c(1024) + c(512), c(512), n(2), True)
        self.h16 = C3k2(c(512) + c(512), c(256), n(2), True)                 # salida P3
        self.h17 = Conv(c(256), c(256), 3, 2)
        self.h19 = C3k2(c(256) + c(512), c(512), n(2), True)                 # salida P4
        self.h20 = Conv(c(512), c(512), 3, 2)
        self.h22 = C3k2(c(512) + c(1024), c(1024), 1, True, 0.5, atencion=True)  # P5
        self.cabeza = Cabeza(nc, (c(256), c(512), c(1024)))
```

Paso 11 **Escribir `FuegoNet.forward`.** El recorrido se lee de arriba abajo como la figura:

```
    def forward(self, x):
        x = self.b2(self.b1(self.b0(x)))
        p3 = self.b4(self.b3(x))
        p4 = self.b6(self.b5(p3))
        p5 = self.b10(self.b9(self.b8(self.b7(p4))))
        t4 = self.h13(torch.cat((self.sube(p5), p4), 1))     # de arriba abajo
        s3 = self.h16(torch.cat((self.sube(t4), p3), 1))
        s4 = self.h19(torch.cat((self.h17(s3), t4), 1))      # de abajo arriba
        s5 = self.h22(torch.cat((self.h20(s4), p5), 1))
        return self.cabeza([s3, s4, s5])
```

Tabla 4.1: FuegoNet-n capa por capa (entrada 640 × 640, 2 clases).

Capa	Bloque	Entra de	Salida	Parámetros
0	Conv 3×3 s2	imagen	320 × 320 × 16	464
1	Conv 3×3 s2	0	160 × 160 × 32	4 672
2	C3k2 ($e = 0.25$)	1	160 × 160 × 64	6 640
3	Conv 3×3 s2	2	80 × 80 × 64	36 992
4	C3k2 ($e = 0.25$)	3	80 × 80 × 128	26 080
5	Conv 3×3 s2	4	40 × 40 × 128	147 712
6	C3k2 (C3k)	5	40 × 40 × 128	87 040
7	Conv 3×3 s2	6	20 × 20 × 256	295 424
8	C3k2 (C3k)	7	20 × 20 × 256	346 112
9	SPPF	8	20 × 20 × 256	164 608
10	C2PSA	9	20 × 20 × 256	249 728
13	C3k2	sube(10) + 6	40 × 40 × 128	119 808
16	C3k2	sube(13) + 4	80 × 80 × 64	34 304
17	Conv 3×3 s2	16	40 × 40 × 64	36 992
19	C3k2	17 + 13	40 × 40 × 128	95 232
20	Conv 3×3 s2	19	20 × 20 × 128	147 712
22	C3k2 + atención	20 + 10	20 × 20 × 256	463 104
23	Cabeza (o2m + o2o)	16, 19, 22	300 × 6	241 956
	Total			**2 504 580**

4.4. La cabeza: de mapas a cajas

Las tres salidas del cuello tienen $80^2 + 40^2 + 20^2 = 8\,400$ celdas. Cada celda tiene un centro (a_x, a_y) y un **paso** s (8, 16 o 32 píxeles): el tamaño que ocupa en la imagen original. La cabeza predice para cada celda:

- 4 distancias (ℓ, t, r, b) en unidades de celda, del centro a los lados izquierdo, superior, derecho e inferior de la caja;
- 2 *logits*, uno por clase; la probabilidad es $\sigma(\text{logit})$.

La caja en píxeles es

$$x_1 = s\,(a_x - \ell), \quad y_1 = s\,(a_y - t), \quad x_2 = s\,(a_x + r), \quad y_2 = s\,(a_y + b). \tag{4.1}$$

Para curiosos (te lo puedes saltar)

YOLOv8 y YOLO11 predicen cada distancia como una distribución de probabilidad sobre 16 valores enteros (DFL) y toman su media. YOLO26 la quita (`reg_max=1`): la capa final da 4 números en lugar de 64. El error de las distancias se mide con L1 (capítulo 6).

Paso 12 **Escribir la clase `Cabeza`.** Hay dos ramas por escala: la de cajas (dos Conv 3×3 y una 1×1 que da 4 canales) y la de clases (convoluciones en profundidad, más ligeras). Y todo esto aparece **dos veces**: la cabeza «uno a muchos» (o2m) y su copia «uno a uno» (o2o).

```
    def forward(self, x):
        if not self.training:   # al inferir solo se usa la o2o: la o2m ni se calcula
            return self.decodificar(self._ramas(x, self.caja_o2o, self.clase_o2o))
        o2m = self._ramas(x, self.caja, self.clase)
        # La o2o ve las características desconectadas del grafo: su gradiente
        # no modifica la columna vertebral (la entrena solo la rama o2m).
        o2o = self._ramas([xi.detach() for xi in x], self.caja_o2o, self.clase_o2o)
        return {'o2m': o2m, 'o2o': o2o}
```

En entrenamiento se devuelven las dos ramas para calcular la pérdida. En inferencia solo se calcula la o2o y se decodifica (la o2m queda como peso muerto):

```
    def decodificar(self, p):
        puntos, paso = anclas(p['mapas'], self.pasos)
        cajas = dist_a_caja(p['dist'], puntos) * paso          # ecuación de la caja
        conf, clase = p['logits'].sigmoid().max(-1)
        conf, idx = conf.topk(min(self.max_det, conf.shape[1]), dim=1)
        ...
        return torch.cat((cajas, conf[..., None], clase[..., None]), -1)  # (B, 300, 6)
```

4.5. ¿Por qué ya no hace falta NMS?

Con la asignación clásica, varias celdas vecinas aprenden a detectar el mismo foco de fuego y todas dan una caja parecida. Hay que borrar los duplicados con **NMS** (*Non-Maximum Suppression*): ordenar por confianza y quitar las cajas que traslapan mucho con una mejor. El NMS es un ciclo con

condiciones que no se paraleliza bien, su costo depende del número de cajas y complica exportar la red.

La cabeza o2o se entrena de modo que *solo una celda* por objeto tenga confianza alta (capítulo 5). A la salida basta tomar las 300 puntuaciones más altas y aplicar un umbral. La salida tiene siempre la misma forma $(B, 300, 6)$ y el grafo exportado a ONNX termina ahí.

Paso 13 **Probar la red completa.**

```
~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.modelo
```

Lo que debes ver

```
FuegoNet-n: 2.50 M parámetros, salida (1, 300, 6)
FuegoNet-s: 9.95 M parámetros, salida (1, 300, 6)
FuegoNet-m: 21.78 M parámetros, salida (1, 300, 6)
```

El YOLO26n ajustado de la simulación tiene 2.53 M parámetros. La diferencia (1.3 %) viene de detalles menores de la cabeza. La arquitectura es la misma.

Nota

La inicialización importa. `iniciar_sesgos` pone el sesgo de la última capa de clases en $\log(5/(n_c\,(640/s)^2))$, una probabilidad inicial de alrededor de 0.0001 por celda: al principio la red cree que *todo* es fondo, que es lo correcto en el 99.9 % de las celdas. Sin esto, la pérdida de clase de las primeras iteraciones es enorme y el entrenamiento se vuelve inestable.

Parte III

Enseñarle a la red

Capítulo 5

Quién responde por cada objeto: TAL y STAL

En este capítulo

Antes de calcular el error hay que decidir qué celdas *deberían* haber detectado cada foco de fuego y cuáles son fondo. Esa decisión, la **asignación de etiquetas**, es lo que distingue a un detector de otro. Aquí se programa la de YOLO26: TAL con dos variantes (uno a muchos y uno a uno) y el ajuste STAL para objetos diminutos. Archivo: `red_fuego/asignador.py`.

5.1. El problema

Una imagen produce 8 400 predicciones y tiene, digamos, una caja de fuego y una de humo. ¿Contra qué etiqueta comparamos cada predicción? Si a todas las celdas dentro de la caja les exigimos «fuego», las del borde (que apenas ven llamas) reciben un objetivo imposible. Si solo a la del centro, la red recibe muy poca señal.

Ejemplo de la vida diaria

Un maestro reparte la tarea de describir un cuadro entre sus alumnos. No le pide a todo el salón que lo describa, ni a uno solo al azar: escoge a los que ya están mirando el cuadro y lo describen mejor. TAL hace eso: premia a las celdas que ya van por buen camino.

5.2. TAL: asignación alineada a la tarea

Para cada etiqueta g y cada celda i cuyo centro cae dentro de la caja de g, se calcula la **métrica de alineación**

$$t_{ig} = s_{ig}^{\alpha}\,\mathrm{IoU}_{ig}^{\beta}, \qquad \alpha = 0.5,\ \beta = 6, \tag{5.1}$$

donde s_{ig} es la probabilidad que la celda i da a la clase de g y IoU_{ig} es el traslape (CIoU, capítulo 6) entre su caja predicha y la caja de g. Con $\beta = 6$ el traslape pesa mucho más que la clase: una celda solo «gana» si su caja ya es buena.

El algoritmo, para cada imagen:

1. **Candidatas**: celdas con el centro dentro de la caja g.
2. **Métrica**: t_{ig} en las candidatas (cero fuera).
3. **Top-k**: por cada g, las k celdas con mayor t_{ig} son positivas.
4. **Conflictos**: si una celda quedó positiva para dos etiquetas, se queda con la de mayor IoU.
5. **Objetivo suave**: el objetivo de clase de una celda positiva no es 1, sino $\hat{t}_{ig} = t_{ig} \dfrac{\max_j \mathrm{IoU}_{jg}}{\max_j t_{jg}}$.
   La mejor celda de cada objeto apunta a su IoU y las demás a menos. Así la confianza aprende a reflejar *qué tan buena* es la caja.

5.3. Uno a muchos y uno a uno

YOLO26 usa el mismo asignador con dos configuraciones, una para cada cabeza:

	Uno a muchos (o2m)	**Uno a uno (o2o)**
Top-k	10	7 y luego 1
Positivos por objeto	hasta 10	exactamente 1
Para qué	mucha señal, aprende rápido	sin duplicados ⇒ sin NMS
Se usa al inferir	no	sí

La o2o primero toma las 7 mejores celdas (como la o2m) y, ya resueltos los conflictos, se queda solo con la de mayor t_{ig} (`topk2=1`). Esa celda aprende a dar confianza alta y sus vecinas aprenden a dar confianza baja aunque vean el mismo fuego. Eso es justo lo que hacía el NMS, pero ahora lo aprende la red.

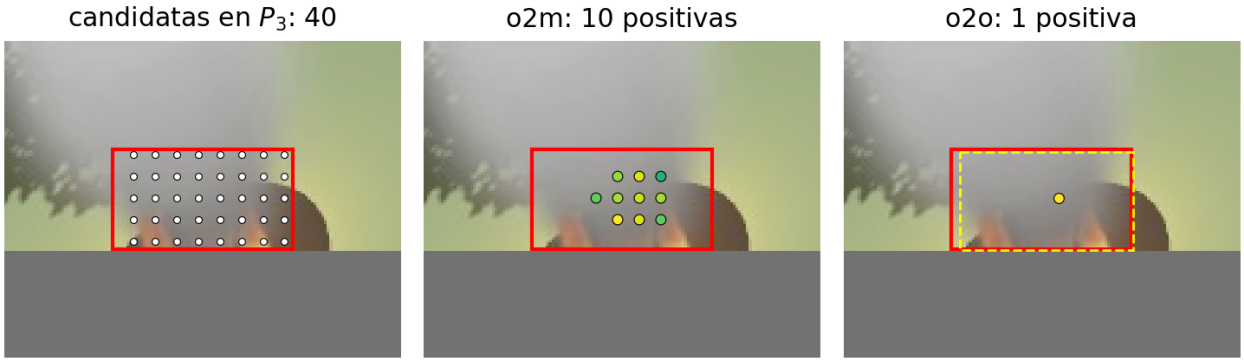


Figura 5.1: Asignación real en una imagen de validación con FuegoNet ya entrenada. Izquierda: celdas candidatas de la caja de fuego (puntos grises) en la escala P_3 (paso 8). Centro: las 10 celdas positivas de la cabeza o2m (color = métrica t). Derecha: la única celda positiva de la o2o y, punteada, la caja que predice.

5.4. STAL: que ningún foco se quede sin responsable

Una celda es candidata solo si su *centro* cae dentro de la caja. En la escala más fina los centros están cada 8 píxeles. Un foco visto desde 60 m puede medir 6×5 píxeles y caer entre cuatro centros sin

contener ninguno. Esa etiqueta queda sin celdas positivas, la red nunca aprende de ella y el error pasa sin que se note.

STAL (*Small-Target-Aware Label assignment*) resuelve esto. Si un lado de la caja mide menos que el paso intermedio (16 píxeles), al buscar candidatas se agranda a 16 píxeles alrededor de su centro. Solo se agranda para elegir candidatas: el objetivo de la regresión sigue siendo la caja original.

```
    def _dentro(self, puntos, cajas):
        c = (cajas[..., :2] + cajas[..., 2:]) / 2
        wh = (cajas[..., 2:] - cajas[..., :2]).clamp(min=self.lado_min)      # STAL
        lt, rb = (c - wh / 2)[:, :, None], (c + wh / 2)[:, :, None]
        d = torch.cat((puntos - lt, rb - puntos), -1)                         # (B, M, A, 4)
        return d.amin(-1) > self.eps
```

5.5. El código

Paso 14 **Escribir `AsignadorTAL`.** Todo se hace con tensores de forma (B, M, A): imágenes, etiquetas (rellenadas con ceros hasta el máximo del lote) y celdas. No hay ciclos de Python sobre celdas ni sobre etiquetas:

```
        dentro = self._dentro(puntos, cajas_gt) & valido.bool()               # (B, M, A)
        b, m, a = dentro.nonzero(as_tuple=True)
        iou[b, m, a] = ciou(cajas_gt[b, m], cajas_p[b, a]).clamp_(0)
        s[b, m, a] = prob[b, a, clases_gt[b, m, 0].long()]
        t = s.pow(self.alpha) * iou.pow(self.beta)
        idx = t.topk(self.topk, dim=-1).indices                                # (B, M, k)
        pos = torch.zeros_like(t, dtype=torch.int8).scatter_(-1, idx, 1)
        pos = pos * dentro * valido.to(torch.int8)
        varios = (pos.sum(1, keepdim=True) > 1).expand(-1, M, -1)             # conflictos
        mejor = torch.zeros_like(pos).scatter_(1, iou.argmax(1, keepdim=True), 1)
        pos = torch.where(varios, mejor, pos)
        if self.topk2 != self.topk:                                           # uno a uno
            idx2 = (t * pos).topk(self.topk2, dim=-1).indices
            pos = pos * torch.zeros_like(pos).scatter_(-1, idx2, 1)
```

Cuidado

El asignador va decorado con `@torch.no_grad()` y recibe las predicciones con `.detach()`. La asignación *usa* la predicción para decidir, pero no se deriva a través de esa decisión. Si se deriva, la red aprende a hacer trampa con su propia asignación.

Pruébalo tú

En `perdida.py` cambia `lado_min` a 0 (STAL apagado), entrena 30 épocas y compara el AP de fuego con el del modelo normal en las imágenes del mundo grande, donde los focos se ven más pequeños.

Capítulo 6

La función de pérdida

En este capítulo

Un solo número que diga qué tan mal lo hizo la red: clase, caja y distancias, en dos cabezas que se combinan con un peso que cambia (ProgLoss). Archivos: `red_fuego/perdida.py` y `red_fuego/perdida_iou.py`.

6.1. Tres términos por cabeza

Una vez asignadas las celdas, cada cabeza calcula

$$L = 7.5\, L_{\text{caja}} + 0.5\, L_{\text{clase}} + 1.5\, L_{\text{dist}}, \tag{6.1}$$

con las ganancias por omisión de Ultralytics. Sea Ω el conjunto de celdas positivas, $\hat{t}_i$ el objetivo suave de la celda i (capítulo 5) y $S = \sum_i \hat{t}_i$.

6.1.1. Clase: entropía cruzada binaria

En *todas* las celdas y para cada clase c:

$$L_{\text{clase}} = \frac{1}{S} \sum_{i} \sum_{c} -\left[\hat{t}_{ic} \log \sigma(z_{ic}) + (1 - \hat{t}_{ic}) \log(1 - \sigma(z_{ic}))\right]. \tag{6.2}$$

Las celdas de fondo tienen $\hat{t} = 0$ y aprenden a decir «nada». Se usa una sigmoide por clase (no un *softmax*) porque fuego y humo pueden aparecer en la misma celda.

6.1.2. Caja: CIoU

Paso 15 **Escribir `ciou`.** El IoU mide el traslape entre la caja predicha B y la real B^*, pero vale cero cuando no se tocan y entonces no da gradiente. CIoU (*Complete IoU*) le resta dos castigos:

$$\text{CIoU} = \text{IoU} - \frac{\rho^2(\mathbf{c}, \mathbf{c}^*)}{d^2} - \alpha v, \qquad v = \frac{4}{\pi^2}\Big(\arctan\frac{w^*}{h^*} - \arctan\frac{w}{h}\Big)^2, \tag{6.3}$$

donde ρ es la distancia entre los centros, d la diagonal de la caja más pequeña que contiene a las dos y $\alpha = v/(1 - \mathrm{IoU} + v)$. El primer castigo acerca los centros aunque las cajas no se toquen; el segundo iguala la proporción ancho/alto. La pérdida es

$$L_{\text{caja}} = \frac{1}{S} \sum_{i \in \Omega} \hat{t}_i \, (1 - \mathrm{CIoU}_i). \tag{6.4}$$

6.1.3. Distancias: L1 (en lugar de DFL)

Las cuatro distancias (ℓ, t, r, b) se comparan directamente con las reales, pasadas a píxeles y divididas entre el tamaño de la imagen para que todas estén entre 0 y 1:

$$L_{\text{dist}} = \frac{1}{S} \sum_{i \in \Omega} \hat{t}_i \, \frac{1}{4} \sum_{k \in \{\ell, t, r, b\}} \left| \frac{s_i \, d_{ik}}{640} - \frac{s_i \, d^*_{ik}}{640} \right|. \tag{6.5}$$

Es el término que reemplaza a la DFL de YOLOv8 y YOLO11.

6.2. Dos cabezas y ProgLoss

Paso 16 **Escribir `PerdidaE2E`.** La pérdida total combina las dos cabezas:

$$L_{\text{total}} = w(e) \, L_{\text{o2m}} + (1 - w(e)) \, L_{\text{o2o}}, \qquad w(e) = 0.1 + 0.7 \left(1 - \frac{e}{E - 1}\right), \tag{6.6}$$

donde e es la época actual y E el total. Al principio ($w = 0.8$) manda la cabeza o2m: con diez positivos por objeto, la columna vertebral aprende rápido a ver fuego. Al final ($w = 0.1$) manda la o2o, que es la que se usará en vuelo.

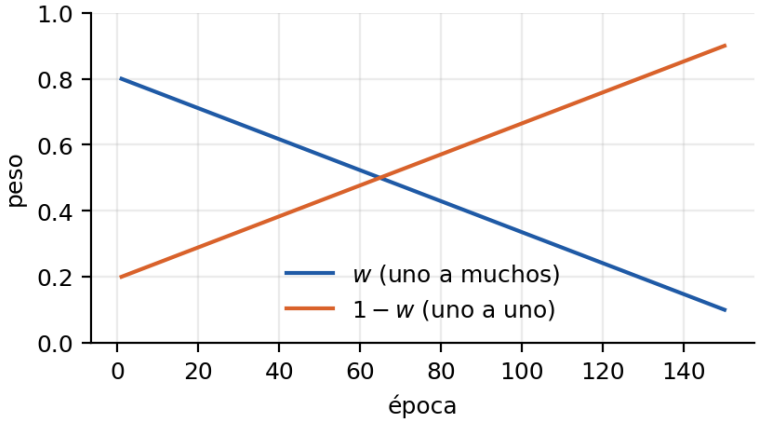


Figura 6.1: Peso de cada cabeza a lo largo de 150 épocas (ProgLoss).

```python
class PerdidaE2E:
    def __init__(self, nc, pasos, epocas, w0=0.8, wf=0.1):
        self.o2m = PerdidaCabeza(nc, pasos, topk=10)
        self.o2o = PerdidaCabeza(nc, pasos, topk=7, topk2=1)
        ...
    def fijar_epoca(self, e):
        self.w = max(1 - e / max(self.epocas - 1, 1), 0) * (self.w0 - self.wf) + self.wf
```

Recuerda

Recuerda el `detach()` de la cabeza (capítulo 4): la pérdida o2o solo entrena las capas propias de esa cabeza. La columna vertebral y el cuello los entrena únicamente la o2m. Por eso la o2o puede volverse «selectiva» sin estorbar a las características que comparten.

6.3. Detalles que importan

- La pérdida se calcula en `float32` aunque la red corra en `bfloat16` (precisión mixta): los logaritmos y las sumas grandes pierden precisión en 16 bits.
- Se multiplica por el tamaño del lote B (como Ultralytics) para que la escala del gradiente no dependa de B cuando se acumulan lotes.
- Si una imagen del lote no tiene objetos, solo contribuye a L_{clase} (todo es fondo). Si *ningún* objeto del lote tiene celdas positivas, L_{caja} y L_{dist} valen cero pero siguen conectadas al grafo, así `backward()` no falla.

Capítulo 7

Leer los datos y aumentarlos

En este capítulo

Cómo se cargan las imágenes y sus etiquetas, cómo se llevan a 640×640 sin deformarlas (letterbox) y qué aumentos se aplican para que 5 400 imágenes alcancen. Archivo: `red_fuego/datos.py`.

7.1. De archivo a tensor

Paso 17 **Escribir `DatasetYolo`.** Un `Dataset` de PyTorch solo necesita dos métodos: `__len__` (cuántas muestras hay) y `__getitem__(i)` (dame la muestra i). El `DataLoader` los usa para armar lotes en paralelo con varios procesos (`num_workers=8`).

Cada muestra devuelve la imagen como tensor `uint8` $(3, 640, 640)$ en RGB y una tabla de etiquetas $(n, 5)$ con `clase,cx,cy,w,h` normalizados. La función `juntar` une un lote y agrega a cada etiqueta el índice de su imagen, porque cada imagen tiene un número distinto de objetos.

7.2. Letterbox: cambiar de tamaño sin deformar

La cámara da 640×480 y la red quiere 640×640. Estirar la imagen deformaría las llamas. **Letterbox** la escala para que el lado mayor mida 640 y rellena el resto con gris (valor 114). Para esta cámara: escala $r = 1$ y 80 píxeles de gris arriba y abajo. Al inferir hay que deshacerlo: $x_{\text{orig}} = (x - d_x)/r$, $y_{\text{orig}} = (y - d_y)/r$.

7.3. Aumentos de datos

El simulador tiene la luz siempre igual, los mismos árboles y el mismo fuego. Si la red solo ve eso, aprende detalles que no se repiten en el mundo real (el *sobreajuste*). Los aumentos crean variantes nuevas en cada época:

Mosaico Cuatro imágenes en un lienzo de 1280×1280, alrededor de un centro al azar; luego se recorta a 640×640. La red ve objetos partidos, a varias escalas y con contextos mezclados. Es

el aumento que más ayuda a YOLO.

Afín Escala al azar entre 0.5 y 1.5 y traslación de ±10 %. Simula volar más alto o más bajo. Las cajas se transforman con la misma matriz y se descartan las que quedan casi fuera (menos de 2 píxeles o menos del 10 % de su área).

HSV Tono ±1.5 %, saturación ±50 % y brillo ±40 % con tablas de búsqueda (rápido). Simula otra hora del día u otra cámara.

Espejo Horizontal con probabilidad 0.5.

Paso 18 **Ver los aumentos antes de entrenar.** Este es el paso más importante del capítulo: si las cajas no coinciden con el fuego después de aumentar, la red aprende basura y el error no aparece en ningún lado.

```
from cuadrirotor_sim.red_fuego.datos import DatasetYolo
ds = DatasetYolo('.../data.yaml', 'train', 640, aumentar=True)
img, e = ds[3000]                    # dibuja e sobre img y guárdala
```

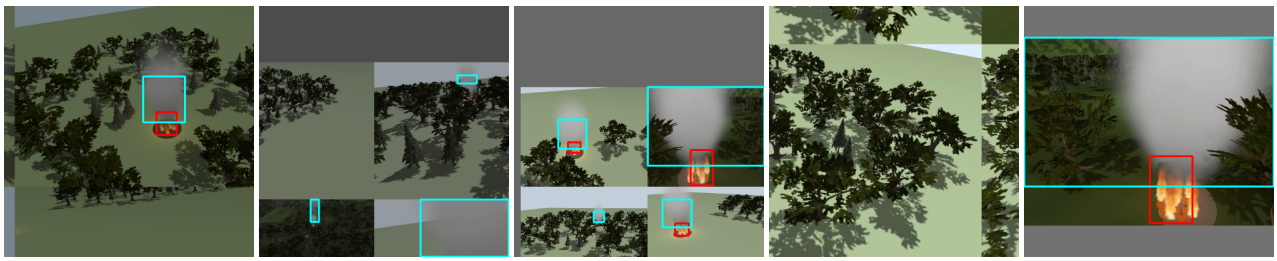


Figura 7.1: Cuatro muestras aumentadas (mosaico + afín + HSV + espejo) y una de validación sin aumentar (derecha). Rojo: fuego. Cian: humo.

Lo que debes ver

Cada caja roja encierra llamas y cada caja cian una columna de humo, incluso en los pedazos del mosaico. Las franjas grises son el relleno.

Consejo

En las últimas 10 épocas se apaga el mosaico (`--sin-mosaico10`). Así la red termina viendo imágenes como las que verá en vuelo. Como los procesos del `DataLoader` guardan su propia copia del dataset, al cambiar `ds.mosaico` hay que crear un `DataLoader` nuevo.

Problema encontrado y cómo se resolvió

Las etiquetas de las imágenes sin fuego son archivos vacíos y `np.loadtxt` da un aviso por cada uno (más de mil líneas en la terminal). Se revisa el tamaño del archivo antes de leerlo:

```
    if not os.path.exists(p) or os.path.getsize(p) == 0:   # imagen sin fuego ni humo
        return np.zeros((0, 5), np.float32)
```

Capítulo 8

Entrenar

En este capítulo

El optimizador MuSGD, la tasa de aprendizaje con calentamiento, el promedio móvil de pesos (EMA) y el ciclo que junta todo. Al final, el comando para entrenar FuegoNet-n desde cero y lo que pasó en las 150 épocas. Archivos: `red_fuego/optimizador.py` y `red_fuego/entrenar.py`.

8.1. El optimizador MuSGD

Entrenar es repetir: calcular la pérdida de un lote, obtener el gradiente $\mathbf{g} = \nabla L$ con `backward()` y mover los pesos un poco en contra. El descenso por gradiente estocástico con momento (SGD) hace

$$\mathbf{m} \leftarrow \mu\,\mathbf{m} + \mathbf{g}, \qquad \mathbf{W} \leftarrow \mathbf{W} - \eta\,(\mathbf{g} + \mu\,\mathbf{m}) \quad \text{(Nesterov)}. \tag{8.1}$$

Muon trata cada matriz de pesos como un todo. Una convolución $c_{\text{out}} \times c_{\text{in}} \times k \times k$ se ve como matriz $\mathbf{G}$ de $c_{\text{out}} \times (c_{\text{in}}k^2)$. Si $\mathbf{G} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ (descomposición en valores singulares), Muon cambia la actualización por $\mathbf{U}\mathbf{V}^\top$: la misma dirección, pero con *todos* los valores singulares iguales a 1. Ninguna dirección domina a las demás y las «débiles» también avanzan.

Calcular la SVD en cada paso sería lentísimo. Muon aproxima $\mathbf{U}\mathbf{V}^\top$ con cinco iteraciones de Newton-Schulz, solo con multiplicaciones de matrices:

$$\mathbf{X}_0 = \frac{\mathbf{G}}{\|\mathbf{G}\|_F}, \qquad \mathbf{A} = \mathbf{X}\mathbf{X}^\top, \qquad \mathbf{X} \leftarrow a\,\mathbf{X} + (b\,\mathbf{A} + c\,\mathbf{A}^2)\,\mathbf{X}, \tag{8.2}$$

con $a = 3.4445$, $b = -4.7750$ y $c = 2.0315$.

Paso 19 **Escribir `ortogonalizar` y `MuSGD`.**

```
def ortogonalizar(G, pasos=5, eps=1e-7):
    a, b, c = 3.4445, -4.7750, 2.0315
    X = G.bfloat16()
    X = X / (X.norm() + eps)                # valor singular mayor <= 1
    alta = G.size(0) > G.size(1)
    if alta:
        X = X.T
    for _ in range(pasos):
        A = X @ X.T
```

```
        X = a * X + (b * A + c * A @ A) @ X
    return (X.T if alta else X).to(G.dtype)
```

YOLO26 aplica a los pesos de convolución las *dos* actualizaciones, Muon con factor 0.2 y SGD con factor 1:

$$\mathbf{W} \leftarrow \mathbf{W} - 0.2\,\eta\,\mathrm{Muon}(\mathbf{g})\sqrt{\max(1, c_{\mathrm{out}}/c_{\mathrm{in}}k^2)} - \eta\,\mathrm{SGD}(\mathbf{g} + \lambda\mathbf{W}), \tag{8.3}$$

y a los sesgos y a BatchNorm solo SGD, sin decaimiento de pesos ($\lambda = 0$).

8.2. Tasa de aprendizaje, calentamiento y lote nominal

- **Ley lineal**: $\eta(e) = \eta_0\,[(1 - e/E)(1 - 0.01) + 0.01]$, de $\eta_0 = 0.01$ a 0.0001.
- **Calentamiento**: en las primeras 3 épocas η sube desde 0 (los sesgos bajan desde 0.1) y el momento de 0.8 a 0.9. Con pesos al azar, un paso grande al principio puede arruinar las estadísticas de BatchNorm.
- **Lote nominal 64**: con lotes de 32 se acumulan dos gradientes antes de cada paso. El decaimiento de pesos se escala por $32 \cdot 2/64 = 1$.
- **Recorte**: la norma del gradiente se limita a 10.

8.3. EMA: el modelo suavizado

Se guarda una copia de la red cuyos pesos son un promedio móvil exponencial de los pesos de entrenamiento:

$$\mathbf{W}_{\mathrm{EMA}} \leftarrow d\,\mathbf{W}_{\mathrm{EMA}} + (1 - d)\,\mathbf{W}, \qquad d = 0.9999\,(1 - e^{-n/2000}), \tag{8.4}$$

donde n es el número de pasos dados. La EMA ignora el «temblor» de cada lote. Es la que se valida y se guarda.

8.4. El ciclo de entrenamiento

Paso 20 **Escribir el ciclo.** Lo esencial de `entrenar.py`:

```
for epoca in range(a.epocas):
    perdida.fijar_epoca(epoca)                      # ProgLoss
    for k, lote in enumerate(carga_ent):
        ...                                         # calentamiento de lr y momento
        lote['img'] = lote['img'].to(disp).float() / 255
        with torch.autocast('cuda', torch.bfloat16):
            pred = modelo(lote['img'])              # {'o2m': ..., 'o2o': ...}
        l, partes = perdida(pred, lote)
        (l / acumular).backward()
        if (k + 1) % acumular == 0:
            torch.nn.utils.clip_grad_norm_(modelo.parameters(), 10.0)
            opt.step(); opt.zero_grad(set_to_none=True)
            ema.actualizar(modelo)
    m = validar(ema.m, carga_val, nc, disp)         # mAP en las 600 de validación
    # guarda ultimo.pt y, si 0.1 mAP50 + 0.9 mAP50-95 mejoró, mejor.pt
```

Nota

`torch.autocast` con `bfloat16` hace las convoluciones en 16 bits (más rápido, la mitad de memoria) y deja en 32 bits lo delicado. A diferencia de `float16`, `bfloat16` tiene el mismo rango que `float32`, así que no necesita `GradScaler`. Las tarjetas RTX de la serie 30 en adelante lo soportan.

Paso 21 **Primera prueba: tres épocas.** Antes de lanzar horas de entrenamiento se corre una prueba corta. Si hay un error en la asignación o en la pérdida, el mAP se queda en cero:

```
~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.entrenar \
  --datos ~/datasets/combinado_incendio_bosque_incendio_bosque_grande/data.yaml \
  --epocas 3 --salida /tmp/prueba_fuegonet
```

Lo que debes ver

```
FuegoNet-n: 2.50 M parámetros, 5400 imágenes de entrenamiento, 600 de validación
[  1/3] pérdida caja 3.364 clase 8.700 dist 0.045 | mAP50 0.001 mAP50-95 0.000 | 74 s
[  2/3] pérdida caja 3.004 clase 4.437 dist 0.042 | mAP50 0.077 mAP50-95 0.022 | 122 s
[  3/3] pérdida caja 2.507 clase 2.656 dist 0.034 | mAP50 0.512 mAP50-95 0.198 | 171 s
```

Las tres pérdidas bajan y el mAP50 sube de 0 a 0.5 en tres épocas, sin ningún peso preentrenado. La red está aprendiendo.

Paso 22 **Entrenamiento completo.**

```
nohup ~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.entrenar \
  --datos ~/datasets/combinado_incendio_bosque_incendio_bosque_grande/data.yaml \
  --epocas 150 --salida ~/datasets/fuegonet/n_150 > ~/datasets/fuegonet/n_150.log 2>&1 &
tail -f ~/datasets/fuegonet/n_150.log
```

Cada época queda en una fila de `resultados.csv`: tiempo, las tres pérdidas de la o2o, P, R, mAP50, mAP50-95, AP por clase, tasa de aprendizaje y peso w de ProgLoss.

8.5. Qué pasó en el entrenamiento

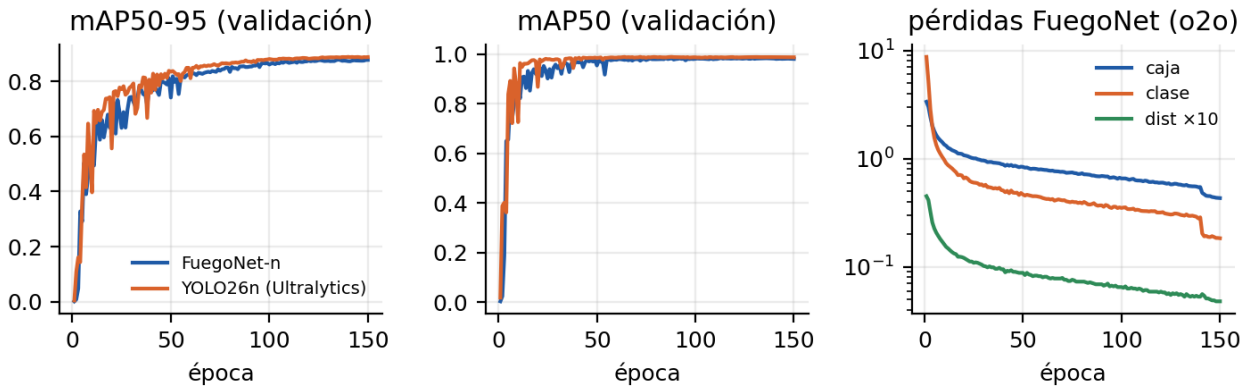


Figura 8.1: Entrenamiento desde cero. Izquierda y centro: mAP en validación de FuegoNet-n (este manual) y de YOLO26n de Ultralytics con la misma receta. Derecha: pérdidas de la cabeza o2o de FuegoNet (escala logarítmica).

- El entrenamiento corrió 150 épocas en 156 minutos (unos 62 s por época, compartiendo la GPU con la corrida de control).
- El mejor modelo (`mejor.pt`) es el de la época 135: mAP50 = 0.984 y mAP50-95 = 0.879 en validación (AP50-95 de fuego 0.879, de humo 0.879).
- En la época 10 ya tenía mAP50 = 0.859: la mayor parte se aprende pronto y las últimas épocas afinan las cajas (sube sobre todo el mAP50-95).
- Al apagar el mosaico (últimas 10 épocas) hay un salto pequeño hacia arriba: la red se ajusta a imágenes completas como las de vuelo.

Nota

La curva naranja es el control: YOLO26n de Ultralytics, también desde cero, con los mismos datos y aumentos. Su mejor época (140) da mAP50-95 = 0.889 *según su propio validador*. La comparación justa, con el mismo código de métricas, está en el capítulo 9.

Parte IV

Medir y usar la red

Capítulo 9

Medir y comparar con YOLO26n

En este capítulo

Cómo se calcula el mAP, por qué hay que medir todas las redes con el mismo código y cómo queda FuegoNet-n frente a YOLO26n entrenado desde cero y frente al YOLO26n preentrenado que usa hoy la simulación. Archivos: `red_fuego/metricas.py` y `red_fuego/evaluar.py`.

9.1. mAP en pocas palabras

Para un umbral de traslape τ, una predicción es un **verdadero positivo** si su IoU con una etiqueta de su misma clase es $\geq \tau$ y esa etiqueta no la tomó antes otra predicción con más confianza. Ordenando todas las predicciones de la clase por confianza se obtienen la precisión $P = \mathrm{VP}/(\mathrm{VP} + \mathrm{FP})$ y la exhaustividad (*recall*) $R = \mathrm{VP}/N_{\text{etiquetas}}$ para cada corte. El **AP** es el área bajo la curva $P(R)$ (con su envolvente decreciente, integrada en 101 puntos, como COCO).

- **mAP50**: AP con $\tau = 0.5$, promediado en las dos clases. ¿Encontró el fuego?
- **mAP50-95**: promedio en $\tau = 0.50, 0.55, \ldots, 0.95$. ¿Qué tan ajustada está la caja? Es la métrica principal.

Paso 23 **Escribir el acumulador de métricas.** `Acumulador.agregar(pred,gt)` recibe, imagen por imagen, las predicciones $(N, 6)$ y las etiquetas $(M, 5)$, y guarda qué predicciones fueron VP en cada uno de los 10 umbrales. `calcular()` ordena todo por confianza y devuelve mAP50, mAP50-95, AP por clase, y P y R con confianza ≥ 0.25 (el umbral del detector en vuelo).

Cuidado

Ultralytics reporta su propio mAP, pero calcula la validación con imágenes rectangulares agrupadas, su propio emparejamiento y su propio preprocesamiento. Comparar su número con el nuestro sería comparar dos reglas distintas. `evaluar.py` corre las *tres* redes sobre las mismas 600 imágenes originales (640×480), con umbral de confianza 0.001 y hasta 300 detecciones, y las mide con el *mismo* `Acumulador`.

9.2. Las tres redes que se comparan

FuegoNet-n nuestra red, desde cero, 150 épocas (capítulo 8).

YOLO26n desde cero la red de Ultralytics con la misma arquitectura (`yolo26n.yaml`, `pretrained=False`), los mismos datos, aumentos y épocas. Es el **control**: si nuestro código está bien, debe salir parecida a FuegoNet-n.

YOLO26n preentrenado el de la simulación (`yolo/incendio_yolo26n.pt`): pesos de COCO ajustados 60 épocas a fuego y humo.

Paso 24 **Correr la evaluación.**

```
~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.evaluar \
  --datos ~/datasets/combinado_incendio_bosque_incendio_bosque_grande/data.yaml \
  --fuegonet ~/datasets/fuegonet/n_150/mejor.pt \
  --yolo ~/datasets/fuegonet/yolo26n_desde_cero/weights/best.pt yolo/incendio_yolo26n.pt \
  --salida ~/datasets/fuegonet/evaluacion.json
```

9.3. Resultados

Tabla 9.1: Las tres redes medidas con el mismo código sobre las 600 imágenes de validación. P y R con confianza ≥ 0.25 e IoU 0.5. Mediana de la latencia por imagen (lote 1, incluye pre y posprocesamiento) en la RTX 5060 Ti y en CPU con 4 hilos.

Red	Parám. (M)	GFLOPs	P	R	mAP50	mAP50-95	AP fuego	AP humo	GPU (ms)	CPU (ms)
FuegoNet-n	2.50	5.3	0.918	0.971	0.984	0.899	0.912	0.886	4.0	32.6
YOLO26n desde cero	2.50	5.7	0.893	0.981	0.984	0.888	0.904	0.872	3.8	15.5
YOLO26n preentrenado (COCO)	2.53	5.9	0.912	0.973	0.984	0.881	0.895	0.867	4.0	21.1

FuegoNet-n contra YOLO26n desde cero. Es la comparación que valida el código: misma arquitectura, mismos datos, mismas épocas y sin preentrenamiento. Las dos tienen los mismos 2.505 M de parámetros: es la misma red. FuegoNet-n hace 5.3 GFLOPs y no 5.7 porque al inferir ya no calcula la cabeza o2m, que Ultralytics sí calcula aunque no la use. FuegoNet-n queda por encima en 0.011 de mAP50-95 (0.899 contra 0.888). Es una sola corrida de cada una: para decir que una es mejor habría que repetir con varias semillas. Lo que sí se puede concluir es que la red propia *reproduce* a YOLO26n. Además, el validador de Ultralytics le da 0.889 a su modelo y nuestro código 0.888, así que también el código de métricas coincide.

Contra el YOLO26n preentrenado. El modelo que usa hoy la simulación da mAP50-95 = 0.881, 0.018 por debajo de FuegoNet-n. Puede sorprender, porque partió de pesos de COCO (118 000 imágenes). Pero se ajustó solo 60 épocas y FuegoNet entrenó 150, y las imágenes de validación vienen del mismo simulador que las de entrenamiento. En este caso, más épocas sobre datos del dominio pesan más que el preentrenamiento. Con imágenes reales de incendios, donde COCO sí aporta, el resultado podría invertirse (capítulo 11). En mAP50 las tres redes empatan (0.984): todas encuentran el fuego. La diferencia está en qué tan ajustadas quedan las cajas.

Velocidad. En la GPU las tres tardan entre 4 y 4 ms por imagen (mediana de 100), contando el pre y el posprocesamiento: unas 250 imágenes por segundo. En CPU, FuegoNet-n tarda 33 ms y los

YOLO26n entre 16 y 21 ms. Aunque es la misma red, Ultralytics infiere con la imagen *rectangular* de 640 × 480 y FuegoNet con el cuadrado de 640 × 640 (25 % más píxeles). Con `Detector(...,rect=True)` FuegoNet baja a unos 23 ms, pero su mAP50-95 cae de 0.899 a 0.886 porque se entrenó solo con imágenes cuadradas. Por eso, por omisión, se usa el cuadrado. Los tiempos en CPU varían algunos milisegundos de una medición a otra.

Problema encontrado y cómo se resolvió

Durante el entrenamiento, el mejor modelo (época 135) marcaba mAP50-95 = 0.879 y aquí da 0.899. La diferencia viene de la precisión: `entrenar.py` valida con `bfloat16` (rápido) y `evaluar.py` en `float32`. Con 8 bits de mantisa, una coordenada entre 512 y 1024 píxeles se redondea de 4 en 4 píxeles, y eso castiga los umbrales de IoU altos (0.90, 0.95). Se comprobó validando el mismo `mejor.pt` de las dos formas: 0.880 en `bfloat16` y 0.898 en `float32`. Para reportar resultados, usa siempre `evaluar.py`. Queda pendiente decodificar las cajas en `float32` dentro de `Cabeza.decodificar`: así la elección del mejor modelo también usaría la métrica exacta.

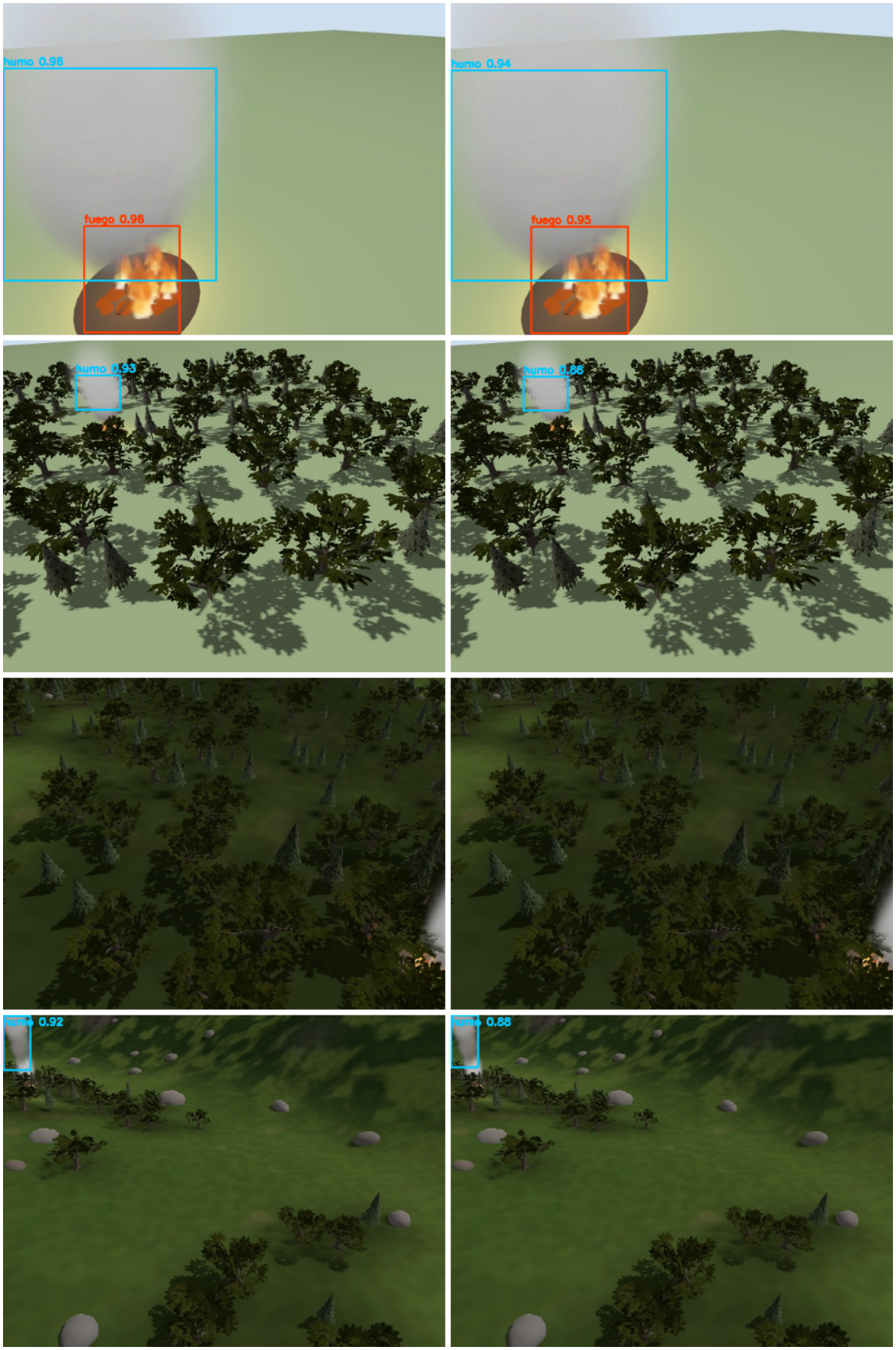


Figura 9.1: Detecciones con confianza ≥ 0.25 en cuatro imágenes de validación. Izquierda: FuegoNet-n (sin NMS). Derecha: YOLO26n preentrenado de la simulación.

Capítulo 10

Usar la red: imágenes, ONNX y ROS 2

En este capítulo

Detectar en una foto, exportar la red a ONNX para la computadora a bordo y conectarla a la simulación para que el Tarot 650 busque el incendio con FuegoNet. Archivos: `red_fuego/inferir.py`, `red_fuego/exportar.py`, `detector_yolo.py` y `launch/simulacion.launch.py`.

10.1. Detectar en una imagen

Paso 25 **Probar el detector.** La clase `Detector` carga `mejor.pt`, aplica letterbox, corre la red, aplica el umbral de confianza (no hay NMS) y regresa las cajas a las coordenadas de la imagen original.

Al cargar los pesos, `fusionar` pliega cada BatchNorm en su convolución. Al inferir, BatchNorm es solo una transformación lineal fija por canal:

$$y = \gamma\,\frac{Wx-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta = \underbrace{\frac{\gamma}{\sqrt{\sigma^2+\epsilon}}W}_{W'}\,x + \underbrace{\beta - \frac{\gamma\,\mu}{\sqrt{\sigma^2+\epsilon}}}_{b'}, \tag{10.1}$$

así que se puede meter dentro de los pesos de la convolución. Da exactamente las mismas cajas (el mAP no cambia) con una operación menos por capa.

```
~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.inferir \
  ~/datasets/fuegonet/n_150/mejor.pt ~/datasets/incendio_bosque/images/val/00420.jpg salida.jpg
```

Lo que debes ver

```
2 detecciones en 6.1 ms
  fuego  0.96  caja 117.2 322.9 255.2 476.0
  humo   0.96  caja 0.0 94.8 308.8 401.7
```

Cada fila es una caja: clase, confianza y (x_1, y_1, x_2, y_2) en píxeles de la imagen original de 640×480.

10.2. Exportar a ONNX

ONNX es un formato abierto que entienden TensorRT (Jetson), OpenVINO (Intel), ONNX Runtime (cualquier CPU) y muchos otros. Como FuegoNet no tiene NMS, todo el detector cabe en el grafo y la salida es un solo tensor $(1, 300, 6)$.

Paso 26 **Instalar `onnx` sin romper NumPy.**

```
~/.venvs/yolo/bin/pip install onnx onnxruntime
~/.venvs/yolo/bin/pip install "numpy==1.26.4" "ml-dtypes<0.6"   # ¡regresar NumPy!
```

Problema encontrado y cómo se resolvió

Al instalar `onnx`, pip subió NumPy de 1.26 a 2.5 y `ml-dtypes` a una versión que exige NumPy 2. ROS 2 Jazzy está compilado contra NumPy 1 y el nodo detector habría dejado de arrancar. La segunda línea regresa NumPy y fija una `ml-dtypes` compatible. Comprueba siempre con `python-c"importnumpy;print(numpy.__version__)"` después de instalar algo.

Paso 27 **Exportar y comprobar.**

```
~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.exportar \
  ~/datasets/fuegonet/n_150/mejor.pt ~/datasets/fuegonet/fuegonet_n.onnx \
  ~/datasets/incendio_bosque/images/val/00420.jpg
```

El tercer argumento es una foto para comprobar que ONNX Runtime y PyTorch dan las mismas detecciones.

Lo que debes ver

```
Exportado /home/jairo/datasets/fuegonet/fuegonet_n.onnx
2 detecciones con confianza >= 0.25; diferencia máxima ONNX vs PyTorch: 3.05e-05
```

El archivo pesa 9.6 MB. La entrada se llama `imagen` $(1, 3, 640, 640)$ y la salida `detecciones` $(1, 300, 6)$.

Problema encontrado y cómo se resolvió

La primera comprobación usaba una entrada de ruido y comparaba las 300 filas. Daba diferencias de más de 500 píxeles. No era un error de exportación: con ruido casi todas las confianzas son ≈ 0 y están empatadas, y `topk` las ordena distinto en PyTorch y en ONNX Runtime, así que se comparaban cajas de celdas distintas. Ahora se usa una foto real y solo las detecciones con confianza ≥ 0.25. La diferencia baja a 3×10^{-5} píxeles.

10.3. Conectar FuegoNet a la simulación

El nodo `detector_yolo` ya publicaba `/cuadrirotor/detecciones` (`vision_msgs/Detection2DArray`) con YOLO26 de Ultralytics. Para no duplicar el nodo, `inferir.py` incluye una envoltura `ComoYolo` con la misma interfaz que usa el nodo (`.names` y `.predict(...)[0].boxes`). El nodo elige según la terminación del archivo de pesos:

```
        if v['modelo'].endswith('.fuegonet.pt'):
            from cuadrirotor_sim.red_fuego.inferir import ComoYolo
            self.modelo, self.etiqueta = ComoYolo(v['modelo'], v['dispositivo']), 'FuegoNet'
        else:
            self.modelo, self.etiqueta = YOLO(v['modelo']), 'YOLO26'
```

Paso 28 **Instalar los pesos en el paquete.**

```
cp ~/datasets/fuegonet/n_150/mejor.pt \
   ~/ros2_ws/src/cuadrirotor_sim/yolo/incendio_fuegonet_n.fuegonet.pt
cd ~/ros2_ws && colcon build --packages-select cuadrirotor_sim --symlink-install
source install/setup.bash
```

Paso 29 **Volar la búsqueda con FuegoNet.** El *launch* tiene un argumento nuevo, `red`, que elige el detector (`yolo26`, el de siempre, o `fuegonet`):

```
ros2 launch cuadrirotor_sim simulacion.launch.py mision:=busqueda red:=fuegonet
```

Lo que debes ver

En la terminal aparece `YOLOlisto:incendio_fuegonet_n.fuegonet.pt,clases\{0:'fuego',1:'humo'\}` y el panel «Detección YOLO» de Gazebo muestra `FuegoNetxxms` en la esquina. El resto de la misión (barrido, verificación, aviso GPS) no cambia, porque los mensajes son los mismos.

Consejo

Para comparar las dos redes en las mismas condiciones, corre la misión dos veces con el mismo `mundo`, `foco_x`, `foco_y` y `viento`, una con `red:=yolo26` y otra con `red:=fuegonet`, y compara el tiempo hasta la confirmación y las falsas alarmas.

Capítulo 11

Qué sigue

En este capítulo

Resumen de lo construido, cómo reportarlo en un artículo o reporte y experimentos que valen la pena con FuegoNet.

11.1. Lo que ya tienes

- Una implementación propia, legible y sin dependencias externas de la arquitectura y la receta de entrenamiento de YOLO26: bloques C3k2, SPPF y C2PSA, cabeza sin DFL ni NMS, TAL con STAL, pérdida de dos cabezas con ProgLoss y optimizador MuSGD.
- Un modelo entrenado desde cero, medido con un protocolo propio y comparado contra la implementación de referencia.
- La red conectada a la misión de búsqueda (`red:=fuegonet`) y exportable a ONNX.

11.2. Cómo reportarlo

1. En el marco teórico, describe la arquitectura con la figura 4.1 y la tabla 4.1, y la asignación y la pérdida con las ecuaciones de los capítulos 5 y 6.
2. En la metodología, da el dataset (tabla 2.1), los aumentos, los hiperparámetros del capítulo 8 y el protocolo de medición (mismo código para todas las redes).
3. En los resultados, la tabla comparativa del capítulo 9 y las curvas de entrenamiento. Separa con claridad «arquitectura igual» de «pesos iniciales distintos».
4. Cita YOLO26 por su documentación de Ultralytics, YOLOv10 por la idea de la doble asignación sin NMS, TOOD por TAL y el trabajo de Muon por el optimizador.

11.3. Experimentos sugeridos

Ablación Quitar una pieza a la vez y medir: sin STAL (`lado_min=0`), sin atención (`atencion=False` en `h22` y sin `C2PSA`), SGD en lugar de MuSGD (`--optimizadorsgd`), sin ProgLoss (w fijo en

0.5). Cada pieza debe justificar su lugar.

Escala Entrenar FuegoNet-s (`--escalas`, 10 M parámetros) y ver si el aumento de mAP compensa la latencia en la computadora a bordo.

Preentrenamiento propio Entrenar primero con un dataset público de incendios reales (por ejemplo D-Fire) y después ajustar con el simulador. En el simulador FuegoNet ya supera al YOLO26n preentrenado, pero con imágenes reales el preentrenamiento suele pesar mucho más.

Varias semillas Repetir FuegoNet-n y el YOLO26n desde cero con 3 semillas (`--semilla`) y reportar media y desviación. Solo así se puede afirmar que una diferencia de 0.01 en mAP50-95 es real.

Fusionar BatchNorm Plegar cada BatchNorm en su convolución y quitar la cabeza o2m antes de inferir, para acercar la latencia en CPU a la de Ultralytics.

Del simulador al mundo real Probar la red con videos reales de incendios tomados con dron. Ahí se verá si los aumentos de color alcanzan o si hace falta aleatorizar más el simulador (texturas, luz, niebla).

Clase «distractor» Agregar una tercera clase para lo que hoy da falsas alarmas (rocas oscuras, claros con sol) en lugar de dejarlo como fondo.

Recuerda

Una red que mide bien en validación todavía no está probada en vuelo. La prueba final es la misión completa: cuánto tarda el Tarot 650 en encontrar y confirmar el foco, y cuántas falsas alarmas da en el camino.

Apéndice A

Código completo

Estos son los archivos de `cuadrirotor_sim/red_fuego/` tal como se usaron para los resultados de este manual.

A.1. `bloques.py`

```
"""Bloques de construcción de FuegoNet, la red tipo YOLO26 para fuego y humo.

Cada bloque es una pieza de la familia YOLO escrita desde cero con PyTorch puro,
sin depender de ultralytics:

  Conv        convolución + BatchNorm + SiLU, la unidad mínima.
  DWConv      convolución en profundidad (un filtro por canal), barata.
  Cuello      «bottleneck» residual: dos Conv 3x3 y un atajo x + f(x).
  C3k         CSP con cuellos de botella de kernel k (dos ramas, una se procesa).
  C3k2        el bloque principal de YOLO11/YOLO26: divide los canales, apila
              n subbloques y concatena todas las salidas intermedias.
  SPPF        agrupamiento piramidal rápido: tres max-pool 5x5 en cadena.
  Atencion    autoatención multicabeza sobre el mapa de características.
  BloquePSA   atención + red de avance con atajos (como un transformador).
  C2PSA       CSP con bloques PSA, al final de la columna vertebral.
"""
import math

import torch
from torch import nn


def relleno(k, d=1):
    """Relleno que conserva el tamaño espacial con kernel k y dilatación d."""
    k = d * (k - 1) + 1 if d > 1 else k
    return k // 2


class Conv(nn.Module):
    """Conv2d sin sesgo + BatchNorm2d + SiLU (o identidad si act=False)."""

    def __init__(self, c1, c2, k=1, s=1, g=1, d=1, act=True):
        super().__init__()
        self.conv = nn.Conv2d(c1, c2, k, s, relleno(k, d), groups=g, dilation=d, bias=False)
```

```
        self.bn = nn.BatchNorm2d(c2, eps=1e-3, momentum=0.03)
        self.act = nn.SiLU(inplace=True) if act else nn.Identity()

    def forward(self, x):
        return self.act(self.bn(self.conv(x)))


class DWConv(Conv):
    """Convolución en profundidad: grupos = máximo común divisor de los canales."""

    def __init__(self, c1, c2, k=3, s=1, act=True):
        super().__init__(c1, c2, k, s, g=math.gcd(c1, c2), act=act)


class Cuello(nn.Module):
    """Bottleneck: y = x + Conv(Conv(x)) si hay atajo y los canales coinciden."""

    def __init__(self, c1, c2, atajo=True, k=(3, 3), e=0.5):
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = Conv(c1, c_, k[0])
        self.cv2 = Conv(c_, c2, k[1])
        self.suma = atajo and c1 == c2

    def forward(self, x):
        y = self.cv2(self.cv1(x))
        return x + y if self.suma else y


class C3k(nn.Module):
    """CSP de tres convoluciones: una rama pasa por n cuellos, la otra no."""

    def __init__(self, c1, c2, n=2, atajo=True, e=0.5, k=3):
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = Conv(c1, c_, 1)
        self.cv2 = Conv(c1, c_, 1)
        self.cv3 = Conv(2 * c_, c2, 1)
        self.m = nn.Sequential(*(Cuello(c_, c_, atajo, (k, k), e=1.0) for _ in range(n)))

    def forward(self, x):
        return self.cv3(torch.cat((self.m(self.cv1(x)), self.cv2(x)), 1))


class C3k2(nn.Module):
    """Bloque C3k2 (C2f con subbloques intercambiables).

    La entrada se proyecta a 2c canales y se parte en dos mitades. La segunda
    mitad pasa por n subbloques en cadena y se guarda cada salida; al final se
    concatenan las dos mitades y las n salidas ((2 + n) c canales) y una Conv 1x1
    las mezcla. Así el gradiente tiene caminos cortos hacia todas las capas.

    El subbloque es un Cuello, un C3k (c3k=True) o un Cuello seguido de
    atención (atencion=True), como en la última etapa de YOLO26.
    """

    def __init__(self, c1, c2, n=1, c3k=False, e=0.5, atencion=False, atajo=True):
        super().__init__()
        self.c = int(c2 * e)
```

```
        self.cv1 = Conv(c1, 2 * self.c, 1)
        self.cv2 = Conv((2 + n) * self.c, c2, 1)
        if atencion:
            sub = lambda: nn.Sequential(Cuello(self.c, self.c, atajo),
                                        BloquePSA(self.c, cabezas=max(self.c // 64, 1)))
        elif c3k:
            sub = lambda: C3k(self.c, self.c, 2, atajo)
        else:
            sub = lambda: Cuello(self.c, self.c, atajo)
        self.m = nn.ModuleList(sub() for _ in range(n))

    def forward(self, x):
        y = list(self.cv1(x).chunk(2, 1))
        for m in self.m:
            y.append(m(y[-1]))
        return self.cv2(torch.cat(y, 1))


class SPPF(nn.Module):
    """Spatial Pyramid Pooling - Fast.

    Tres max-pool 5x5 en cadena equivalen a ventanas de 5, 9 y 13 píxeles: la
    red ve el contexto a varias escalas sin costo de parámetros. Con atajo=True
    (YOLO26) se suma la entrada a la salida.
    """

    def __init__(self, c1, c2, k=5, n=3, atajo=False):
        super().__init__()
        c_ = c1 // 2
        self.cv1 = Conv(c1, c_, 1, act=False)
        self.cv2 = Conv(c_ * (n + 1), c2, 1)
        self.m = nn.MaxPool2d(k, 1, k // 2)
        self.n = n
        self.suma = atajo and c1 == c2

    def forward(self, x):
        y = [self.cv1(x)]
        for _ in range(self.n):
            y.append(self.m(y[-1]))
        y = self.cv2(torch.cat(y, 1))
        return x + y if self.suma else y


class Atencion(nn.Module):
    """Autoatención multicabeza con convoluciones 1x1 y codificación posicional.

    Cada píxel del mapa (N = H*W) es un «token». Se calculan consultas q,
    llaves k y valores v; el peso entre dos píxeles es softmax(q·k/sqrt(dk)).
    Una convolución 3x3 en profundidad sobre v hace de codificación posicional.
    """

    def __init__(self, dim, cabezas=8, razon=0.5):
        super().__init__()
        self.cabezas = cabezas
        self.dc = dim // cabezas            # dimensión de cada cabeza (valores)
        self.dk = int(self.dc * razon)      # dimensión de consultas y llaves
        self.escala = self.dk ** -0.5
        self.qkv = Conv(dim, dim + 2 * self.dk * cabezas, 1, act=False)
        self.proy = Conv(dim, dim, 1, act=False)
```

```
        self.pe = Conv(dim, dim, 3, 1, g=dim, act=False)

    def forward(self, x):
        B, C, H, W = x.shape
        q, k, v = self.qkv(x).view(B, self.cabezas, 2 * self.dk + self.dc, H * W).split(
            [self.dk, self.dk, self.dc], dim=2)
        a = ((q * self.escala).transpose(-2, -1) @ k).softmax(dim=-1)   # (B, h, N, N)
        y = (v @ a.transpose(-2, -1)).view(B, C, H, W) + self.pe(v.reshape(B, C, H, W))
        return self.proy(y)


class BloquePSA(nn.Module):
    """Atención y red de avance (FFN), cada una con su conexión residual."""

    def __init__(self, c, razon=0.5, cabezas=4, atajo=True):
        super().__init__()
        self.atn = Atencion(c, cabezas, razon)
        self.ffn = nn.Sequential(Conv(c, 2 * c, 1), Conv(2 * c, c, 1, act=False))
        self.suma = atajo

    def forward(self, x):
        x = x + self.atn(x) if self.suma else self.atn(x)
        return x + self.ffn(x) if self.suma else self.ffn(x)


class C2PSA(nn.Module):
    """CSP con n bloques PSA: la atención se aplica solo a la mitad de los canales."""

    def __init__(self, c1, c2, n=1, e=0.5):
        super().__init__()
        assert c1 == c2
        self.c = int(c1 * e)
        self.cv1 = Conv(c1, 2 * self.c, 1)
        self.cv2 = Conv(2 * self.c, c1, 1)
        self.m = nn.Sequential(*(BloquePSA(self.c, 0.5, max(self.c // 64, 1)) for _ in range(n)))

    def forward(self, x):
        a, b = self.cv1(x).split((self.c, self.c), dim=1)
        return self.cv2(torch.cat((a, self.m(b)), 1))
```

A.2. modelo.py

```
"""FuegoNet: detector de una etapa con la arquitectura de YOLO26.

Tres partes, igual que YOLO26:
  columna vertebral  (backbone)  extrae características a 1/8, 1/16 y 1/32;
  cuello             (neck, PAN-FPN) mezcla las tres escalas de arriba abajo y
                     de abajo arriba;
  cabeza             (head) predice, en cada celda de cada escala, 4 distancias
                     a los lados de la caja (l, t, r, b) y una puntuación por clase.

Lo propio de YOLO26 que se conserva:
  * reg_max = 1: las distancias se regresan directo, sin la distribución DFL
    de YOLOv8/YOLO11 (más simple y más rápido de exportar).
  * Cabeza doble: «uno a muchos» (o2m) solo para entrenar y «uno a uno» (o2o)
    para inferir. La o2o aprende a dar UNA caja por objeto, así que no hace
```

```
    falta NMS: basta tomar las k puntuaciones más altas.

Las escalas siguen el yolo26.yaml de ultralytics ([profundidad, anchura, máx]).
"""
import copy
import math

import torch
from torch import nn

from .bloques import C2PSA, C3k2, SPPF, Conv, DWConv

ESCALAS = {'n': (0.50, 0.25, 1024), 's': (0.50, 0.50, 1024), 'm': (0.50, 1.00, 512)}
PASOS = (8, 16, 32)


def anclas(mapas, pasos, desfase=0.5):
    """Centros de las celdas (en unidades de celda) y el paso de cada una."""
    puntos, paso = [], []
    for m, s in zip(mapas, pasos):
        _, _, h, w = m.shape
        sy, sx = torch.meshgrid(torch.arange(h, device=m.device, dtype=m.dtype) + desfase,
                                torch.arange(w, device=m.device, dtype=m.dtype) + desfase,
                                indexing='ij')
        puntos.append(torch.stack((sx, sy), -1).view(-1, 2))
        paso.append(torch.full((h * w, 1), s, device=m.device, dtype=m.dtype))
    return torch.cat(puntos), torch.cat(paso)


def dist_a_caja(dist, puntos):
    """(l, t, r, b) respecto al centro de la celda -> (x1, y1, x2, y2)."""
    lt, rb = dist.chunk(2, -1)
    return torch.cat((puntos - lt, puntos + rb), -1)


class Cabeza(nn.Module):
    """Cabeza de detección sin NMS (Detect de YOLO26 con end2end=True)."""

    def __init__(self, nc, canales):
        super().__init__()
        self.nc, self.nl = nc, len(canales)
        self.pasos = PASOS
        c2 = max(16, canales[0] // 4, 4)
        c3 = max(canales[0], min(nc, 100))
        # Rama de cajas: dos Conv 3x3 y una 1x1 que da 4 números (reg_max = 1).
        self.caja = nn.ModuleList(
            nn.Sequential(Conv(x, c2, 3), Conv(c2, c2, 3), nn.Conv2d(c2, 4, 1)) for x in canales)
        # Rama de clases: convoluciones en profundidad + 1x1 (más ligera).
        self.clase = nn.ModuleList(
            nn.Sequential(nn.Sequential(DWConv(x, x, 3), Conv(x, c3, 1)),
                          nn.Sequential(DWConv(c3, c3, 3), Conv(c3, c3, 1)),
                          nn.Conv2d(c3, nc, 1)) for x in canales)
        # Copias para la asignación uno a uno.
        self.caja_o2o = copy.deepcopy(self.caja)
        self.clase_o2o = copy.deepcopy(self.clase)
        self.max_det = 300

    def iniciar_sesgos(self, imgsz=640):
        """Sesgo de clase = log(prior) para que al inicio casi todo sea fondo."""
```

```
        for ramas in ((self.caja, self.clase), (self.caja_o2o, self.clase_o2o)):
            for a, b, s in zip(*ramas, self.pasos):
                a[-1].bias.data[:] = 2.0
                b[-1].bias.data[:] = math.log(5 / self.nc / (imgsz / s) ** 2)

    def _ramas(self, x, caja, clase):
        bs = x[0].shape[0]
        d = torch.cat([caja[i](x[i]).view(bs, 4, -1) for i in range(self.nl)], -1)
        c = torch.cat([clase[i](x[i]).view(bs, self.nc, -1) for i in range(self.nl)], -1)
        return {'dist': d.permute(0, 2, 1), 'logits': c.permute(0, 2, 1), 'mapas': x}

    def forward(self, x):
        if not self.training:   # al inferir solo se usa la o2o: la o2m ni se calcula
            return self.decodificar(self._ramas(x, self.caja_o2o, self.clase_o2o))
        o2m = self._ramas(x, self.caja, self.clase)
        # La rama o2o ve las características desconectadas del grafo: su gradiente
        # no modifica la columna vertebral (la entrena solo la rama o2m).
        o2o = self._ramas([xi.detach() for xi in x], self.caja_o2o, self.clase_o2o)
        return {'o2m': o2m, 'o2o': o2o}

    def decodificar(self, p):
        """Salida (B, max_det, 6): x1, y1, x2, y2 en píxeles, confianza, clase."""
        puntos, paso = anclas(p['mapas'], self.pasos)
        cajas = dist_a_caja(p['dist'], puntos) * paso
        prob = p['logits'].sigmoid()
        conf, clase = prob.max(-1)
        k = min(self.max_det, conf.shape[1])
        conf, idx = conf.topk(k, dim=1)
        cajas = cajas.gather(1, idx[..., None].expand(-1, -1, 4))
        clase = clase.gather(1, idx)
        return torch.cat((cajas, conf[..., None], clase[..., None].to(cajas.dtype)), -1)


class FuegoNet(nn.Module):
    """Columna vertebral + cuello PAN + cabeza sin NMS, como yolo26.yaml."""

    def __init__(self, nc=2, escala='n'):
        super().__init__()
        prof, anch, cmax = ESCALAS[escala]
        c = lambda x: int(math.ceil(min(x, cmax) * anch / 8) * 8)   # canales múltiplos de 8
        n = lambda x: max(round(x * prof), 1)                        # repeticiones
        c3k = escala in 'ml'                                         # como ultralytics
        self.nc, self.escala = nc, escala
        # --- Columna vertebral ------------------------------------------------
        self.b0 = Conv(3, c(64), 3, 2)                               # P1/2
        self.b1 = Conv(c(64), c(128), 3, 2)                          # P2/4
        self.b2 = C3k2(c(128), c(256), n(2), c3k, 0.25)
        self.b3 = Conv(c(256), c(256), 3, 2)                         # P3/8
        self.b4 = C3k2(c(256), c(512), n(2), c3k, 0.25)              # -> P3
        self.b5 = Conv(c(512), c(512), 3, 2)                         # P4/16
        self.b6 = C3k2(c(512), c(512), n(2), True)                   # -> P4
        self.b7 = Conv(c(512), c(1024), 3, 2)                        # P5/32
        self.b8 = C3k2(c(1024), c(1024), n(2), True)
        self.b9 = SPPF(c(1024), c(1024), 5, 3, atajo=True)
        self.b10 = C2PSA(c(1024), c(1024), n(2))                     # -> P5
        # --- Cuello: de arriba abajo (FPN) ------------------------------------
        self.sube = nn.Upsample(None, 2, 'nearest')
        self.h13 = C3k2(c(1024) + c(512), c(512), n(2), True)
        self.h16 = C3k2(c(512) + c(512), c(256), n(2), True)         # salida P3
```

```
        # --- Cuello: de abajo arriba (PAN) ----------------------------------
        self.h17 = Conv(c(256), c(256), 3, 2)
        self.h19 = C3k2(c(256) + c(512), c(512), n(2), True)             # salida P4
        self.h20 = Conv(c(512), c(512), 3, 2)
        self.h22 = C3k2(c(512) + c(1024), c(1024), 1, True, 0.5, atencion=True)  # P5
        # --- Cabeza ---------------------------------------------------------
        self.cabeza = Cabeza(nc, (c(256), c(512), c(1024)))
        self.cabeza.iniciar_sesgos()

    def forward(self, x):
        x = self.b2(self.b1(self.b0(x)))
        p3 = self.b4(self.b3(x))
        p4 = self.b6(self.b5(p3))
        p5 = self.b10(self.b9(self.b8(self.b7(p4))))
        t4 = self.h13(torch.cat((self.sube(p5), p4), 1))
        s3 = self.h16(torch.cat((self.sube(t4), p3), 1))
        s4 = self.h19(torch.cat((self.h17(s3), t4), 1))
        s5 = self.h22(torch.cat((self.h20(s4), p5), 1))
        return self.cabeza([s3, s4, s5])


def contar_parametros(modelo):
    return sum(p.numel() for p in modelo.parameters())


if __name__ == '__main__':
    for e in ESCALAS:
        m = FuegoNet(2, e).eval()
        with torch.no_grad():
            y = m(torch.zeros(1, 3, 640, 640))
        print(f'FuegoNet-{e}: {contar_parametros(m) / 1e6:.2f} M parámetros, salida {tuple(y.shape)
    }')
```

A.3. asignador.py

```
"""Asignación de etiquetas alineada a la tarea (TAL) con el ajuste STAL de YOLO26.

En cada imagen hay A celdas (anclas) y M objetos etiquetados. Para entrenar hay
que decidir qué celdas «responden» por cada objeto. TAL usa la propia
predicción de la red:

    t = s^alpha * IoU^beta

donde s es la probabilidad que la red da a la clase correcta y IoU es el
traslape de su caja con la etiqueta. Por cada objeto se toman las topk celdas
con mayor t entre las que caen dentro de la caja. Si una celda queda con dos
objetos, se queda con el de mayor IoU.

  * Uno a muchos (o2m): topk = 10, varias celdas por objeto -> señal abundante.
  * Uno a uno  (o2o):   topk = 7 y luego topk2 = 1, una sola celda por objeto
    -> la red aprende a no duplicar cajas y no hace falta NMS.

STAL (Small-Target-Aware Label assignment): una caja con un lado menor que el
paso intermedio (16 px) se agranda a 16 px SOLO para buscar candidatas. Sin
esto, una brasa de 6 px puede no contener ningún centro de celda y nunca
recibir gradiente.
```

```
"""
import torch

from .perdida_iou import ciou


class AsignadorTAL:

    def __init__(self, nc, topk=10, topk2=None, alpha=0.5, beta=6.0, lado_min=16.0, eps=1e-9):
        self.nc, self.topk, self.topk2 = nc, topk, topk2 or topk
        self.alpha, self.beta, self.lado_min, self.eps = alpha, beta, lado_min, eps

    @torch.no_grad()
    def __call__(self, prob, cajas_p, puntos, clases_gt, cajas_gt, valido):
        """
        prob      (B, A, C)  probabilidades predichas (sigmoide)
        cajas_p   (B, A, 4)  cajas predichas xyxy en píxeles
        puntos    (A, 2)     centros de las celdas en píxeles
        clases_gt (B, M, 1)  clase de cada etiqueta
        cajas_gt  (B, M, 4)  cajas etiquetadas xyxy en píxeles
        valido    (B, M, 1)  1 si la etiqueta existe (el lote se rellena con ceros)

        Devuelve cajas_obj (B, A, 4), puntaje_obj (B, A, C) y primer_plano (B, A).
        """
        B, A, _ = prob.shape
        M = cajas_gt.shape[1]
        if M == 0:
            return torch.zeros_like(cajas_p), torch.zeros_like(prob), torch.zeros(B, A, dtype=torch
    .bool,
                                                                                   device=prob.device)
        # 1) Candidatas: centros dentro de la caja (agrandada si es diminuta: STAL).
        dentro = self._dentro(puntos, cajas_gt) & valido.bool()              # (B, M, A)
        # 2) Métrica de alineación solo en las candidatas.
        b, m, a = dentro.nonzero(as_tuple=True)
        iou = torch.zeros(B, M, A, device=prob.device, dtype=cajas_p.dtype)
        iou[b, m, a] = ciou(cajas_gt[b, m], cajas_p[b, a]).clamp_(0).to(iou.dtype)
        s = torch.zeros_like(iou)
        s[b, m, a] = prob[b, a, clases_gt[b, m, 0].long()].to(s.dtype)
        t = s.pow(self.alpha) * iou.pow(self.beta)
        # 3) Las topk mejores celdas de cada objeto.
        idx = t.topk(self.topk, dim=-1).indices                               # (B, M, k)
        pos = torch.zeros_like(t, dtype=torch.int8).scatter_(-1, idx, 1)
        pos = pos * dentro * valido.to(torch.int8)
        # 4) Celda con varios objetos: gana el de mayor IoU.
        varios = (pos.sum(1, keepdim=True) > 1).expand(-1, M, -1)
        mejor = torch.zeros_like(pos).scatter_(1, iou.argmax(1, keepdim=True), 1)
        pos = torch.where(varios, mejor, pos)
        # 5) Uno a uno: de lo que quedó, solo la mejor celda de cada objeto.
        if self.topk2 != self.topk:
            idx2 = (t * pos).topk(self.topk2, dim=-1).indices
            pos = pos * torch.zeros_like(pos).scatter_(-1, idx2, 1)
        primer_plano, obj = pos.max(1)                                        # (B, A)
        # 6) Objetivos de cada celda.
        base = torch.arange(B, device=prob.device)[:, None] * M
        cajas_obj = cajas_gt.view(-1, 4)[obj + base]
        clase_obj = clases_gt.long().view(-1)[obj + base].clamp_(0)
        puntaje = torch.zeros(B, A, self.nc, device=prob.device)
        puntaje.scatter_(2, clase_obj[..., None], 1.0)
        puntaje *= primer_plano[..., None]
```

```
        # 7) Puntaje suave: t normalizado para que la mejor celda de cada objeto
        #    valga su IoU. La red aprende que la confianza debe reflejar la calidad.
        t = t * pos
        iou = iou * pos
        norm = (t * iou.amax(-1, keepdim=True) / (t.amax(-1, keepdim=True) + self.eps)).amax(1)
        return cajas_obj, puntaje * norm[..., None], primer_plano.bool()

    def _dentro(self, puntos, cajas):
        c = (cajas[..., :2] + cajas[..., 2:]) / 2
        wh = (cajas[..., 2:] - cajas[..., :2]).clamp(min=self.lado_min)      # STAL
        lt, rb = (c - wh / 2)[:, :, None], (c + wh / 2)[:, :, None]
        d = torch.cat((puntos - lt, rb - puntos), -1)                        # (B, M, A, 4)
        return d.amin(-1) > self.eps
```

A.4. perdida_iou.py

```
"""IoU y CIoU entre cajas xyxy (mismo número de cajas en a y b)."""
import math

import torch


def iou(a, b, eps=1e-7):
    lt = torch.maximum(a[..., :2], b[..., :2])
    rb = torch.minimum(a[..., 2:], b[..., 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    area_a = (a[..., 2:] - a[..., :2]).prod(-1)
    area_b = (b[..., 2:] - b[..., :2]).prod(-1)
    return inter / (area_a + area_b - inter + eps)


def ciou(a, b, eps=1e-7):
    """Complete IoU: IoU - (distancia entre centros)^2 / diagonal^2 - alpha*v.

    El segundo término jala los centros aunque las cajas no se toquen (el IoU
    solo daría cero y ningún gradiente); el tercero iguala la proporción ancho/alto.
    """
    (ax1, ay1, ax2, ay2), (bx1, by1, bx2, by2) = a.unbind(-1), b.unbind(-1)
    w1, h1 = ax2 - ax1, (ay2 - ay1).clamp(min=eps)
    w2, h2 = bx2 - bx1, (by2 - by1).clamp(min=eps)
    i = iou(a, b, eps)
    cw = torch.maximum(ax2, bx2) - torch.minimum(ax1, bx1)
    ch = torch.maximum(ay2, by2) - torch.minimum(ay1, by1)
    c2 = cw ** 2 + ch ** 2 + eps
    rho2 = ((bx1 + bx2 - ax1 - ax2) ** 2 + (by1 + by2 - ay1 - ay2) ** 2) / 4
    v = (4 / math.pi ** 2) * (torch.atan(w2 / h2) - torch.atan(w1 / h1)) ** 2
    with torch.no_grad():
        alfa = v / (v - i + (1 + eps))
    return i - (rho2 / c2 + v * alfa)
```

A.5. perdida.py

```
"""Función de pérdida de FuegoNet (la de YOLO26, sin DFL).
```

```
Para cada cabeza (o2m y o2o):
    L = 7.5 * L_caja + 0.5 * L_clase + 1.5 * L_dist
  L_clase  entropía cruzada binaria entre los logits y el puntaje suave de TAL,
           en TODAS las celdas (las de fondo deben dar cero).
  L_caja   1 - CIoU en las celdas de primer plano, pesada por su puntaje.
  L_dist   L1 entre las distancias (l, t, r, b) predichas y las reales,
           normalizadas por el tamaño de la imagen (sustituye a la DFL).

Las dos cabezas se combinan con un peso que cambia durante el entrenamiento
(ProgLoss de YOLO26):
    L_total = w * L_o2m + (1 - w) * L_o2o,   w: 0.8 -> 0.1 lineal en las épocas.
Al principio manda la o2m (muchas celdas positivas, aprende rápido); al final
manda la o2o, que es la que se usa para inferir.
"""
import torch
import torch.nn.functional as F

from .asignador import AsignadorTAL
from .modelo import anclas, dist_a_caja
from .perdida_iou import ciou


class PerdidaCabeza:

    def __init__(self, nc, pasos, topk, topk2=None, g_caja=7.5, g_clase=0.5, g_dist=1.5):
        self.nc, self.pasos = nc, pasos
        self.asignador = AsignadorTAL(nc, topk, topk2, lado_min=float(pasos[1]))
        self.g = (g_caja, g_clase, g_dist)

    def __call__(self, pred, etiquetas, imgsz):
        dist, logits = pred['dist'].float(), pred['logits'].float()
        B = dist.shape[0]
        puntos, paso = anclas(pred['mapas'], self.pasos)
        cajas_p = dist_a_caja(dist, puntos)                          # en celdas
        clases_gt, cajas_gt, valido = etiquetas
        cajas_obj, puntaje, fp = self.asignador(
            logits.detach().sigmoid(), (cajas_p.detach() * paso), puntos * paso,
            clases_gt, cajas_gt, valido)
        suma = puntaje.sum().clamp(min=1)
        l_clase = F.binary_cross_entropy_with_logits(logits, puntaje, reduction='none').sum() /
    suma
        l_caja = l_dist = dist.sum() * 0.0
        if fp.any():
            peso = puntaje[fp].sum(-1)
            obj = cajas_obj / paso                                   # a celdas
            l_caja = ((1.0 - ciou(cajas_p[fp], obj[fp])) * peso).sum() / suma
            # Distancias reales en celdas -> píxeles -> fracción de la imagen.
            p = puntos.expand(B, -1, -1)
            dist_obj = torch.cat((p - obj[..., :2], obj[..., 2:] - p), -1)
            esc = paso / imgsz
            l_dist = (F.l1_loss(dist[fp] * esc.expand(B, -1, -1)[fp],
                                dist_obj[fp] * esc.expand(B, -1, -1)[fp], reduction='none')
                      .mean(-1) * peso).sum() / suma
        perdidas = torch.stack((l_caja * self.g[0], l_clase * self.g[1], l_dist * self.g[2]))
        return perdidas.sum() * B, perdidas.detach()


class PerdidaE2E:
```

```
    """o2m + o2o con el peso progresivo de YOLO26."""

    def __init__(self, nc, pasos, epocas, w0=0.8, wf=0.1):
        self.o2m = PerdidaCabeza(nc, pasos, topk=10)
        self.o2o = PerdidaCabeza(nc, pasos, topk=7, topk2=1)
        self.epocas, self.w0, self.wf = epocas, w0, wf
        self.w = w0

    def fijar_epoca(self, e):
        self.w = max(1 - e / max(self.epocas - 1, 1), 0) * (self.w0 - self.wf) + self.wf

    def __call__(self, pred, lote):
        imgsz = lote['img'].shape[-1]
        etiquetas = rellenar(lote, imgsz)
        lm, _ = self.o2m(pred['o2m'], etiquetas, imgsz)
        lo, partes = self.o2o(pred['o2o'], etiquetas, imgsz)
        return self.w * lm + (1 - self.w) * lo, partes


def rellenar(lote, imgsz):
    """Lista de etiquetas (idx, clase, cx, cy, w, h normalizados) -> tensores (B, M, ·) en píxeles.
    """
    t = lote['etiquetas']
    B = lote['img'].shape[0]
    disp = lote['img'].device
    if t.shape[0] == 0:
        return (torch.zeros(B, 0, 1, device=disp), torch.zeros(B, 0, 4, device=disp),
                torch.zeros(B, 0, 1, device=disp))
    i = t[:, 0].long()
    M = int(torch.bincount(i, minlength=B).max())
    clases = torch.zeros(B, M, 1, device=disp)
    cajas = torch.zeros(B, M, 4, device=disp)
    valido = torch.zeros(B, M, 1, device=disp)
    orden = torch.cat([torch.arange(int(n), device=disp) for n in torch.bincount(i, minlength=B)])
    i, t = i.to(disp), t.to(disp)
    xywh = t[:, 2:6] * imgsz
    clases[i, orden, 0] = t[:, 1]
    cajas[i, orden] = torch.cat((xywh[:, :2] - xywh[:, 2:] / 2, xywh[:, :2] + xywh[:, 2:] / 2), -1)
    valido[i, orden, 0] = 1
    return clases, cajas, valido
```

A.6. datos.py

```
"""Lectura del dataset en formato YOLO y aumentos de datos.

Formato: images/{train,val}/xxx.jpg y labels/{train,val}/xxx.txt, una línea por
objeto «clase cx cy w h» normalizados a [0, 1] (lo que escribe
scripts/capturar_dataset.py).

Aumentos de entrenamiento (los mismos que ultralytics por omisión):
  mosaico     cuatro imágenes en un lienzo 2s x 2s alrededor de un centro al azar;
  afín        escala 1 ± 0.5 y traslación ± 10 %;
  HSV         tono ± 0.015, saturación ± 0.5, brillo ± 0.4 (la luz del
              simulador es fija: esto evita que la red dependa del color exacto);
  espejo      horizontal con probabilidad 0.5.
El mosaico se apaga en las últimas épocas para que la red vea imágenes «normales».
```

```
"""
import glob
import os
import random

import cv2
import numpy as np
import torch
import yaml

EXT = ('.jpg', '.jpeg', '.png', '.bmp')


def listar(data_yaml, parte):
    """Rutas de imágenes de 'train' o 'val' (acepta una ruta o una lista)."""
    with open(data_yaml) as f:
        d = yaml.safe_load(f)
    base = d.get('path', os.path.dirname(os.path.abspath(data_yaml)))
    rutas = d[parte] if isinstance(d[parte], list) else [d[parte]]
    imgs = []
    for r in rutas:
        r = r if os.path.isabs(r) else os.path.join(base, r)
        imgs += sorted(p for p in glob.glob(os.path.join(r, '*')) if p.lower().endswith(EXT))
    return imgs, d['names']


def ruta_etiqueta(img):
    a, b = os.sep + 'images' + os.sep, os.sep + 'labels' + os.sep
    return os.path.splitext(b.join(img.rsplit(a, 1)))[0] + '.txt'


def leer_etiquetas(img):
    p = ruta_etiqueta(img)
    if not os.path.exists(p) or os.path.getsize(p) == 0:   # imagen sin fuego ni humo
        return np.zeros((0, 5), np.float32)
    return np.loadtxt(p, ndmin=2, dtype=np.float32).reshape(-1, 5)


def letterbox(img, s, color=114, rect=False):
    """Escala para que el lado mayor mida s y rellena hasta s x s (sin deformar).

    Con rect=True solo rellena hasta el siguiente múltiplo de 32 (el paso mayor de
    la red): una imagen de 640 x 480 queda igual y se procesan 25 % menos píxeles.
    """
    h, w = img.shape[:2]
    r = s / max(h, w)
    nw, nh = round(w * r), round(h * r)
    img = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_LINEAR) if r != 1 else img
    W, H = ((-(-nw // 32) * 32), (-(-nh // 32) * 32)) if rect else (s, s)
    dx, dy = (W - nw) / 2, (H - nh) / 2
    img = cv2.copyMakeBorder(img, round(dy - 0.1), round(dy + 0.1), round(dx - 0.1), round(dx +
    0.1),
                             cv2.BORDER_CONSTANT, value=(color,) * 3)
    return img, r, (dx, dy)


def a_pixeles(e, w, h, dx=0, dy=0):
    """cx cy w h normalizados -> x1 y1 x2 y2 en píxeles (con desplazamiento)."""
    c = e[:, 1:].copy()
```

```
    xy = np.stack((c[:, 0] - c[:, 2] / 2, c[:, 1] - c[:, 3] / 2,
                   c[:, 0] + c[:, 2] / 2, c[:, 1] + c[:, 3] / 2), 1)
    return xy * [w, h, w, h] + [dx, dy, dx, dy]


class DatasetYolo(torch.utils.data.Dataset):

    def __init__(self, data_yaml, parte, imgsz=640, aumentar=False, mosaico=1.0, escala=0.5,
                 traslacion=0.1, hsv=(0.015, 0.5, 0.4), espejo=0.5):
        self.imgs, self.nombres = listar(data_yaml, parte)
        if not self.imgs:
            raise FileNotFoundError(f'No hay imágenes de {parte} en {data_yaml}')
        self.etiquetas = [leer_etiquetas(p) for p in self.imgs]
        self.s, self.aumentar, self.mosaico = imgsz, aumentar, mosaico
        self.escala, self.traslacion, self.hsv, self.espejo = escala, traslacion, hsv, espejo

    def __len__(self):
        return len(self.imgs)

    def _cargar(self, i):
        """Imagen con lado mayor = s y sus cajas xyxy en píxeles de esa imagen."""
        img = cv2.imread(self.imgs[i])
        h, w = img.shape[:2]
        r = self.s / max(h, w)
        if r != 1:
            img = cv2.resize(img, (round(w * r), round(h * r)), interpolation=cv2.INTER_LINEAR)
        h, w = img.shape[:2]
        e = self.etiquetas[i]
        return img, e[:, 0].copy(), a_pixeles(e, w, h)

    def _mosaico(self, i):
        s = self.s
        xc, yc = (int(random.uniform(s * 0.5, s * 1.5)) for _ in range(2))
        lienzo = np.full((2 * s, 2 * s, 3), 114, np.uint8)
        clases, cajas = [], []
        for k, j in enumerate([i] + random.choices(range(len(self)), k=3)):
            img, c, b = self._cargar(j)
            h, w = img.shape[:2]
            if k == 0:   # arriba a la izquierda del centro
                x1a, y1a, x2a, y2a = max(xc - w, 0), max(yc - h, 0), xc, yc
                x1b, y1b, x2b, y2b = w - (x2a - x1a), h - (y2a - y1a), w, h
            elif k == 1:  # arriba a la derecha
                x1a, y1a, x2a, y2a = xc, max(yc - h, 0), min(xc + w, 2 * s), yc
                x1b, y1b, x2b, y2b = 0, h - (y2a - y1a), min(w, x2a - x1a), h
            elif k == 2:  # abajo a la izquierda
                x1a, y1a, x2a, y2a = max(xc - w, 0), yc, xc, min(2 * s, yc + h)
                x1b, y1b, x2b, y2b = w - (x2a - x1a), 0, w, min(y2a - y1a, h)
            else:         # abajo a la derecha
                x1a, y1a, x2a, y2a = xc, yc, min(xc + w, 2 * s), min(2 * s, yc + h)
                x1b, y1b, x2b, y2b = 0, 0, min(w, x2a - x1a), min(y2a - y1a, h)
            lienzo[y1a:y2a, x1a:x2a] = img[y1b:y2b, x1b:x2b]
            clases.append(c)
            cajas.append(b + [x1a - x1b, y1a - y1b, x1a - x1b, y1a - y1b])
        return lienzo, np.concatenate(clases), np.concatenate(cajas), -s // 2

    def _afin(self, img, clases, cajas, borde=0):
        """Escala y traslación al azar; la salida mide s x s."""
        s = self.s
        h, w = img.shape[:2]
```

```
        C = np.eye(3)
        C[0, 2], C[1, 2] = -w / 2, -h / 2                          # centro al origen
        R = np.eye(3)
        R[:2, :2] *= random.uniform(1 - self.escala, 1 + self.escala)
        T = np.eye(3)
        T[0, 2] = random.uniform(0.5 - self.traslacion, 0.5 + self.traslacion) * s
        T[1, 2] = random.uniform(0.5 - self.traslacion, 0.5 + self.traslacion) * s
        M = (T @ R @ C)[:2]
        img = cv2.warpAffine(img, M, (s, s), borderValue=(114, 114, 114))
        if len(cajas):
            esq = np.ones((len(cajas) * 4, 3))
            esq[:, :2] = cajas[:, [0, 1, 2, 3, 0, 3, 2, 1]].reshape(-1, 2)
            esq = (esq @ M.T).reshape(-1, 8)
            nuevas = np.stack((esq[:, 0::2].min(1), esq[:, 1::2].min(1),
                               esq[:, 0::2].max(1), esq[:, 1::2].max(1)), 1).clip(0, s)
            # Descartar cajas que quedaron casi fuera de la imagen.
            area0 = (cajas[:, 2:] - cajas[:, :2]).prod(1) * R[0, 0] ** 2
            wh = nuevas[:, 2:] - nuevas[:, :2]
            ok = (wh > 2).all(1) & (wh.prod(1) / (area0 + 1e-9) > 0.1)
            clases, cajas = clases[ok], nuevas[ok]
        return img, clases, cajas

    def _hsv(self, img):
        g = np.random.uniform(-1, 1, 3) * self.hsv + 1
        h, s, v = cv2.split(cv2.cvtColor(img, cv2.COLOR_BGR2HSV))
        x = np.arange(256, dtype=np.float32)
        lut_h = ((x * g[0]) % 180).astype(np.uint8)
        lut_s = np.clip(x * g[1], 0, 255).astype(np.uint8)
        lut_v = np.clip(x * g[2], 0, 255).astype(np.uint8)
        hsv = cv2.merge((cv2.LUT(h, lut_h), cv2.LUT(s, lut_s), cv2.LUT(v, lut_v)))
        return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    def __getitem__(self, i):
        if self.aumentar and random.random() < self.mosaico:
            img, clases, cajas, _ = self._mosaico(i)
            img, clases, cajas = self._afin(img, clases, cajas)
        else:
            img = cv2.imread(self.imgs[i])
            h, w = img.shape[:2]
            img, r, (dx, dy) = letterbox(img, self.s)
            e = self.etiquetas[i]
            clases, cajas = e[:, 0].copy(), a_pixeles(e, w * r, h * r, dx, dy)
            if self.aumentar:
                img, clases, cajas = self._afin(img, clases, cajas)
        if self.aumentar:
            img = self._hsv(img)
            if random.random() < self.espejo:
                img = np.ascontiguousarray(img[:, ::-1])
                cajas = cajas.copy()
                cajas[:, [0, 2]] = self.s - cajas[:, [2, 0]]
        # Salida: imagen RGB CHW uint8 y etiquetas (clase, cx, cy, w, h) normalizadas.
        cajas = np.asarray(cajas, np.float32).reshape(-1, 4)
        xywh = np.concatenate(((cajas[:, :2] + cajas[:, 2:]) / 2, cajas[:, 2:] - cajas[:, :2]), 1)
    / self.s
        e = np.concatenate((np.asarray(clases, np.float32).reshape(-1, 1), xywh), 1)
        img = torch.from_numpy(np.ascontiguousarray(img[..., ::-1].transpose(2, 0, 1)))
        return img, torch.from_numpy(e)
```

```
def juntar(lote):
    """Une las etiquetas del lote en una tabla (índice de imagen, clase, cx, cy, w, h)."""
    imgs, etqs = zip(*lote)
    filas = [torch.cat((torch.full((len(e), 1), float(i)), e), 1) for i, e in enumerate(etqs)]
    return {'img': torch.stack(imgs), 'etiquetas': torch.cat(filas)}
```

A.7. optimizador.py

```
"""MuSGD: el optimizador de YOLO26 (Muon + SGD con momento de Nesterov).

Muon toma el momento del gradiente de una matriz de pesos G (una convolución
c_out x c_in x k x k se ve como matriz c_out x (c_in k k)) y la reemplaza por
su «parte ortogonal» U V^T (de G = U S V^T). Así todas las direcciones
singulares avanzan con el mismo tamaño de paso y ninguna domina. U V^T se
aproxima con 5 iteraciones de Newton-Schulz, sin calcular la SVD.

En YOLO26 los pesos de convolución reciben las DOS actualizaciones:
    p <- p - lr * 0.2 * Muon(g)  -  lr * 1.0 * SGD(g + wd p)
y los sesgos y BatchNorm solo la de SGD.
"""
import torch
from torch import nn


def ortogonalizar(G, pasos=5, eps=1e-7):
    """Newton-Schulz quíntico: devuelve aproximadamente U V^T con G = U S V^T."""
    a, b, c = 3.4445, -4.7750, 2.0315
    X = G.bfloat16()
    X = X / (X.norm() + eps)                # valor singular mayor <= 1
    alta = G.size(0) > G.size(1)
    if alta:
        X = X.T
    for _ in range(pasos):
        A = X @ X.T
        X = a * X + (b * A + c * A @ A) @ X
    return (X.T if alta else X).to(G.dtype)


class MuSGD(torch.optim.Optimizer):

    def __init__(self, grupos, lr=0.01, momento=0.9, wd=0.0, f_muon=0.2, f_sgd=1.0):
        super().__init__(grupos, dict(lr=lr, momentum=momento, weight_decay=wd, muon=False))
        self.f_muon, self.f_sgd = f_muon, f_sgd

    @torch.no_grad()
    def step(self):
        for g in self.param_groups:
            lr, mu = g['lr'], g['momentum']
            for p in g['params']:
                if p.grad is None:
                    continue
                st = self.state[p]
                if not st:
                    st['m_sgd'] = torch.zeros_like(p)
                    if g['muon']:
                        st['m_muon'] = torch.zeros_like(p)
```

```
                grad = p.grad
                f = 1.0
                if g['muon']:
                    m = st['m_muon'].lerp_(grad, 1 - mu)            # m = mu m + (1-mu) g
                    u = grad.lerp(m, mu)                            # Nesterov
                    u2 = u.view(u.size(0), -1)
                    o = ortogonalizar(u2) * max(1, u2.size(0) / u2.size(1)) ** 0.5
                    p.add_(o.view_as(p), alpha=-lr * self.f_muon)
                    f = self.f_sgd
                d = grad.add(p, alpha=g['weight_decay']) if g['weight_decay'] else grad
                buf = st['m_sgd'].mul_(mu).add_(d)
                p.add_(d.add(buf, alpha=mu), alpha=-lr * f)          # SGD con Nesterov


def grupos_de_parametros(modelo, wd=5e-4, muon=True):
    """Pesos de conv con decaimiento (+Muon), BatchNorm y sesgos sin decaimiento."""
    pesos, bn, sesgos = [], [], []
    for mod in modelo.modules():
        for nombre, p in mod.named_parameters(recurse=False):
            if nombre == 'bias':
                sesgos.append(p)
            elif isinstance(mod, nn.BatchNorm2d):
                bn.append(p)
            else:
                pesos.append(p)
    return [dict(params=pesos, weight_decay=wd, muon=muon),
            dict(params=bn, weight_decay=0.0, muon=False),
            dict(params=sesgos, weight_decay=0.0, muon=False)]
```

A.8. `entrenar.py`

```
#!/usr/bin/env python3
"""Entrena FuegoNet desde cero (sin pesos preentrenados) con el dataset del simulador.

Uso (con el Python de ~/.venvs/yolo):
    ~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.entrenar \
        --datos ~/datasets/combinado_incendio_bosque_incendio_bosque_grande/data.yaml \
        --epocas 150 --salida ~/datasets/fuegonet/n_150

La receta sigue a ultralytics para YOLO26: lote nominal 64 (se acumulan
gradientes si el lote real es menor), calentamiento de 3 épocas, tasa de
aprendizaje lineal de lr0 a lr0*lrf, MuSGD, EMA de los pesos, mosaico apagado en
las últimas 10 épocas y el mejor modelo según 0.1 mAP50 + 0.9 mAP50-95.
"""
import argparse
import csv
import math
import os
import random
import time
from copy import deepcopy

import numpy as np
import torch

from .datos import DatasetYolo, juntar
```

```
from .metricas import Acumulador
from .modelo import FuegoNet, contar_parametros
from .optimizador import MuSGD, grupos_de_parametros
from .perdida import PerdidaE2E


class EMA:
    """Promedio móvil exponencial de los pesos: un modelo «suavizado» que generaliza mejor."""

    def __init__(self, modelo, decaimiento=0.9999, tau=2000):
        self.m = deepcopy(modelo).eval()
        self.n, self.d, self.tau = 0, decaimiento, tau
        for p in self.m.parameters():
            p.requires_grad_(False)

    @torch.no_grad()
    def actualizar(self, modelo):
        self.n += 1
        d = self.d * (1 - math.exp(-self.n / self.tau))
        msd = modelo.state_dict()
        for k, v in self.m.state_dict().items():
            if v.dtype.is_floating_point:
                v.mul_(d).add_(msd[k].detach(), alpha=1 - d)


def cargador(ds, lote, mezclar, trabajadores):
    return torch.utils.data.DataLoader(ds, lote, shuffle=mezclar, num_workers=trabajadores,
                                       collate_fn=juntar, pin_memory=True, drop_last=mezclar,
                                       persistent_workers=trabajadores > 0)


@torch.no_grad()
def validar(modelo, carga, nc, disp, conf=0.001):
    modelo.eval()
    acc = Acumulador(nc)
    for lote in carga:
        img = lote['img'].to(disp, non_blocking=True).float() / 255
        with torch.autocast('cuda', torch.bfloat16, enabled=disp.type == 'cuda'):
            salida = modelo(img).float().cpu().numpy()
        s = img.shape[-1]
        e = lote['etiquetas'].numpy()
        for i, pred in enumerate(salida):
            pred = pred[pred[:, 4] >= conf]
            g = e[e[:, 0] == i]
            xyxy = np.concatenate((g[:, 2:4] - g[:, 4:6] / 2, g[:, 2:4] + g[:, 4:6] / 2), 1) * s
            acc.agregar(pred, np.concatenate((g[:, 1:2], xyxy), 1))
    return acc.calcular()


def principal(a):
    random.seed(a.semilla)
    np.random.seed(a.semilla)
    torch.manual_seed(a.semilla)
    torch.backends.cudnn.benchmark = True
    disp = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    os.makedirs(a.salida, exist_ok=True)

    ds_ent = DatasetYolo(a.datos, 'train', a.imgsz, aumentar=True, mosaico=a.mosaico)
    ds_val = DatasetYolo(a.datos, 'val', a.imgsz)
```

```
    nombres = ds_ent.nombres
    nc = len(nombres)
    carga_ent = cargador(ds_ent, a.lote, True, a.trabajadores)
    carga_val = cargador(ds_val, a.lote * 2, False, a.trabajadores)

    modelo = FuegoNet(nc, a.escala).to(disp)
    if a.pesos:
        modelo.load_state_dict(torch.load(a.pesos, map_location=disp)['modelo'])
    print(f'FuegoNet-{a.escala}: {contar_parametros(modelo) / 1e6:.2f} M parámetros, '
          f'{len(ds_ent)} imágenes de entrenamiento, {len(ds_val)} de validación')

    nominal = 64
    acumular = max(round(nominal / a.lote), 1)
    wd = a.wd * a.lote * acumular / nominal
    grupos = grupos_de_parametros(modelo, wd, muon=a.optimizador == 'musgd')
    if a.optimizador == 'adamw':
        opt = torch.optim.AdamW(grupos, lr=a.lr0, betas=(0.9, 0.999))
    elif a.optimizador == 'sgd':
        opt = torch.optim.SGD(grupos, lr=a.lr0, momentum=0.9, nesterov=True)
    else:
        opt = MuSGD(grupos, lr=a.lr0, momento=0.9)
    for g in opt.param_groups:
        g['lr_inicial'] = a.lr0
    ley = lambda e: (1 - e / a.epocas) * (1.0 - a.lrf) + a.lrf      # lineal lr0 -> lr0*lrf
    perdida = PerdidaE2E(nc, modelo.cabeza.pasos, a.epocas)
    ema = EMA(modelo)

    n_lotes = len(carga_ent)
    n_cal = max(round(3 * n_lotes), 100)            # iteraciones de calentamiento
    mejor, ultima_opt = -1.0, -1
    reg = open(os.path.join(a.salida, 'resultados.csv'), 'w', newline='')
    esc = csv.writer(reg)
    esc.writerow(['epoca', 'tiempo', 'caja', 'clase', 'dist', 'P', 'R', 'mAP50', 'mAP50-95',
                  'AP50-95_fuego', 'AP50-95_humo', 'lr', 'w_o2m'])
    t0 = time.time()
    for epoca in range(a.epocas):
        if epoca == a.epocas - a.sin_mosaico and a.mosaico > 0:
            print('Mosaico apagado para las últimas épocas')
            ds_ent.mosaico = 0.0
            carga_ent = cargador(ds_ent, a.lote, True, a.trabajadores)
        modelo.train()
        perdida.fijar_epoca(epoca)
        suma = torch.zeros(3, device=disp)
        opt.zero_grad(set_to_none=True)
        for k, lote in enumerate(carga_ent):
            it = k + n_lotes * epoca
            if it <= n_cal:   # calentamiento: lr de 0 (sesgos de 0.1) al valor de la ley
                x = it / n_cal
                for j, g in enumerate(opt.param_groups):
                    inicio = 0.1 if (j == 2 and a.optimizador != 'adamw') else 0.0
                    g['lr'] = np.interp(x, [0, 1], [inicio, g['lr_inicial'] * ley(epoca)])
                    if 'momentum' in g:
                        g['momentum'] = np.interp(x, [0, 1], [0.8, 0.9])
            lote['img'] = lote['img'].to(disp, non_blocking=True).float() / 255
            with torch.autocast('cuda', torch.bfloat16, enabled=disp.type == 'cuda'):
                pred = modelo(lote['img'])
            l, partes = perdida(pred, lote)
            (l / acumular).backward()
            suma += partes
```

```
            if (k + 1) % acumular == 0 or k + 1 == n_lotes:
                torch.nn.utils.clip_grad_norm_(modelo.parameters(), 10.0)
                opt.step()
                opt.zero_grad(set_to_none=True)
                ema.actualizar(modelo)
        for g in opt.param_groups:
            g['lr'] = g['lr_inicial'] * ley(epoca + 1)
        m = validar(ema.m, carga_val, nc, disp)
        suma = (suma / n_lotes).tolist()
        apto = 0.1 * m['mAP50'] + 0.9 * m['mAP50-95']
        fila = [epoca + 1, round(time.time() - t0, 1), *[round(x, 4) for x in suma],
                *[round(float(m[x]), 4) for x in ('P', 'R', 'mAP50', 'mAP50-95')],
                *[round(float(x), 4) for x in m['AP50-95_clase']],
                round(opt.param_groups[0]['lr'], 6), round(perdida.w, 3)]
        esc.writerow(fila)
        reg.flush()
        print(f"[{epoca + 1:3d}/{a.epocas}] pérdida caja {suma[0]:.3f} clase {suma[1]:.3f} "
              f"dist {suma[2]:.3f} | mAP50 {m['mAP50']:.3f} mAP50-95 {m['mAP50-95']:.3f} "
              f"| {time.time() - t0:.0f} s", flush=True)
        punto = {'modelo': ema.m.state_dict(), 'nc': nc, 'escala': a.escala, 'nombres': nombres,
                 'epoca': epoca + 1, 'metricas': {k: (v.tolist() if hasattr(v, 'tolist') else v)
                                                  for k, v in m.items()}, 'imgsz': a.imgsz}
        torch.save(punto, os.path.join(a.salida, 'ultimo.pt'))
        if apto > mejor:
            mejor, ultima_opt = apto, epoca
            torch.save(punto, os.path.join(a.salida, 'mejor.pt'))
        if epoca - ultima_opt >= a.paciencia:
            print(f'Sin mejora en {a.paciencia} épocas: se detiene')
            break
    reg.close()
    print(f'Listo en {(time.time() - t0) / 60:.1f} min. Mejor modelo: {os.path.join(a.salida, "
    mejor.pt")}')


def argumentos(args=None):
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.
    RawDescriptionHelpFormatter)
    p.add_argument('--datos', required=True, help='data.yaml del dataset')
    p.add_argument('--salida', required=True, help='carpeta para pesos y resultados.csv')
    p.add_argument('--escala', default='n', choices=['n', 's', 'm'])
    p.add_argument('--epocas', type=int, default=150)
    p.add_argument('--lote', type=int, default=32)
    p.add_argument('--imgsz', type=int, default=640)
    p.add_argument('--optimizador', default='musgd', choices=['musgd', 'sgd', 'adamw'])
    p.add_argument('--lr0', type=float, default=0.01)
    p.add_argument('--lrf', type=float, default=0.01)
    p.add_argument('--wd', type=float, default=5e-4)
    p.add_argument('--mosaico', type=float, default=1.0)
    p.add_argument('--sin-mosaico', type=int, default=10, help='épocas finales sin mosaico')
    p.add_argument('--paciencia', type=int, default=50)
    p.add_argument('--trabajadores', type=int, default=8)
    p.add_argument('--pesos', default='', help='continuar desde un mejor.pt/ultimo.pt')
    p.add_argument('--semilla', type=int, default=0)
    return p.parse_args(args)


if __name__ == '__main__':
    principal(argumentos())
```

A.9. metricas.py

```
"""Precisión media (mAP) como en COCO: mAP50 y mAP50-95.

Para un umbral de IoU t, una predicción es verdadero positivo (VP) si traslapa
con IoU >= t a una etiqueta de su misma clase que nadie más ha tomado (se
recorren de mayor a menor confianza). Con las predicciones ordenadas por
confianza se calculan precisión P = VP / (VP + FP) y exhaustividad
R = VP / (número de etiquetas); AP es el área bajo la curva P(R) con la
envolvente monótona, integrada en 101 puntos. mAP50-95 promedia AP en
t = 0.50, 0.55, ..., 0.95 y en las clases.
"""
import numpy as np

UMBRALES = np.linspace(0.5, 0.95, 10)


def iou_matriz(a, b):
    lt = np.maximum(a[:, None, :2], b[None, :, :2])
    rb = np.minimum(a[:, None, 2:], b[None, :, 2:])
    inter = np.clip(rb - lt, 0, None).prod(2)
    aa = (a[:, 2:] - a[:, :2]).prod(1)
    ab = (b[:, 2:] - b[:, :2]).prod(1)
    return inter / (aa[:, None] + ab[None] - inter + 1e-9)


class Acumulador:
    """Junta las predicciones de todas las imágenes y al final calcula mAP."""

    def __init__(self, nc):
        self.nc = nc
        self.vp, self.conf, self.clase, self.n_gt = [], [], [], np.zeros(nc, int)

    def agregar(self, pred, gt):
        """pred (N, 6) x1 y1 x2 y2 conf clase; gt (M, 5) clase x1 y1 x2 y2 (mismos píxeles)."""
        for c in gt[:, 0].astype(int):
            self.n_gt[c] += 1
        vp = np.zeros((len(pred), len(UMBRALES)), bool)
        if len(pred) and len(gt):
            orden = np.argsort(-pred[:, 4])
            pred = pred[orden]
            iou = iou_matriz(pred[:, :4], gt[:, 1:])
            iou[pred[:, 5][:, None].astype(int) != gt[:, 0][None].astype(int)] = 0
            for k, t in enumerate(UMBRALES):
                tomada = np.zeros(len(gt), bool)
                for i in range(len(pred)):
                    cand = np.where((iou[i] >= t) & ~tomada)[0]
                    if len(cand):
                        j = cand[np.argmax(iou[i, cand])]
                        tomada[j] = vp[i, k] = True
        elif len(pred):
            pred = pred[np.argsort(-pred[:, 4])]
        self.vp.append(vp)
        self.conf.append(pred[:, 4] if len(pred) else np.zeros(0))
        self.clase.append(pred[:, 5].astype(int) if len(pred) else np.zeros(0, int))

    def calcular(self):
        vp, conf, clase = np.concatenate(self.vp), np.concatenate(self.conf), np.concatenate(self.
    clase)
```

```
        ap = np.zeros((self.nc, len(UMBRALES)))
        p_r = np.zeros((self.nc, 2))
        for c in range(self.nc):
            m = clase == c
            if self.n_gt[c] == 0 or not m.any():
                continue
            o = np.argsort(-conf[m])
            v = vp[m][o]
            acum_vp = v.cumsum(0)
            acum_fp = (~v).cumsum(0)
            rec = acum_vp / self.n_gt[c]
            prec = acum_vp / (acum_vp + acum_fp)
            for k in range(len(UMBRALES)):
                ap[c, k] = area_pr(rec[:, k], prec[:, k])
            # P y R al umbral de confianza 0.25 (lo que usa el detector en vuelo)
            sel = conf[m][o] >= 0.25
            if sel.any():
                n = sel.sum() - 1
                p_r[c] = prec[n, 0], rec[n, 0]
        return {'mAP50': ap[:, 0].mean(), 'mAP50-95': ap.mean(), 'AP50-95_clase': ap.mean(1),
                'AP50_clase': ap[:, 0], 'P': p_r[:, 0].mean(), 'R': p_r[:, 1].mean()}


def area_pr(rec, prec):
    r = np.concatenate(([0.0], rec, [1.0]))
    p = np.concatenate(([1.0], prec, [0.0]))
    p = np.flip(np.maximum.accumulate(np.flip(p)))      # envolvente decreciente
    x = np.linspace(0, 1, 101)
    return np.trapz(np.interp(x, r, p), x)
```

A.10. evaluar.py

```
#!/usr/bin/env python3
"""Compara FuegoNet con YOLO26n usando el MISMO código de métricas y las mismas imágenes.

    ~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.evaluar --datos data.yaml \
        --fuegonet mejor.pt --yolo incendio_yolo26n.pt yolo26n_desde_cero.pt --salida eval.json

Mide, en la parte de validación: P, R, mAP50, mAP50-95 por clase, parámetros,
GFLOPs a 640 x 640 y la mediana de la latencia por imagen (lote 1) en GPU y CPU.
"""
import argparse
import json
import os
import time

import cv2
import numpy as np
import torch
from torch.utils.flop_counter import FlopCounterMode

from .datos import a_pixeles, leer_etiquetas, listar
from .inferir import Detector
from .metricas import Acumulador
from .modelo import FuegoNet, contar_parametros
```

```
def gflops(modelo, disp):
    x = torch.zeros(1, 3, 640, 640, device=disp)
    with FlopCounterMode(display=False) as f:
        modelo(x)
    return f.get_total_flops() / 1e9          # una multiplicación-suma cuenta como 2 FLOPs


def latencia(fn, img, n=100):
    """Mediana en ms de n llamadas (la mediana ignora las pausas del sistema)."""
    for _ in range(15):
        fn(img)
    tiempos = []
    for _ in range(n):
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        t0 = time.perf_counter()
        fn(img)
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        tiempos.append(time.perf_counter() - t0)
    return float(np.median(tiempos)) * 1000


def evaluar(nombre, predecir, imgs, nc):
    acc = Acumulador(nc)
    for p in imgs:
        img = cv2.imread(p)
        h, w = img.shape[:2]
        e = leer_etiquetas(p)
        gt = np.concatenate((e[:, :1], a_pixeles(e, w, h)), 1)
        acc.agregar(predecir(img), gt)
    m = acc.calcular()
    print(f"{nombre:28s} P {m['P']:.3f} R {m['R']:.3f} mAP50 {m['mAP50']:.3f} mAP50-95 {m['mAP50
    -95']:.3f}")
    return m


def principal(a):
    imgs, nombres = listar(a.datos, 'val')
    nc = len(nombres)
    muestra = cv2.imread(imgs[0])
    res = {}
    modelos = []
    if a.fuegonet:
        for disp in ('0', 'cpu'):
            d = Detector(a.fuegonet, disp)
            modelos.append(('FuegoNet-' + torch.load(a.fuegonet, map_location='cpu')['escala'],
    disp,
                            lambda img, d=d: d(img, 0.001), d.modelo, d.disp))
    if a.yolo:
        from ultralytics import YOLO
        for ruta in a.yolo:
            for disp in ('0', 'cpu'):
                y = YOLO(ruta)
                f = lambda img, y=y, disp=disp: (lambda r: np.concatenate(
                    (r.boxes.xyxy.cpu().numpy(), r.boxes.conf.cpu().numpy()[:, None],
                     r.boxes.cls.cpu().numpy()[:, None]), 1))(
```

```
                    y.predict(img, conf=0.001, imgsz=640, device=disp, verbose=False, max_det=300)
    [0])
                modelos.append((os.path.basename(ruta), disp, f, y.model, torch.device(
                    'cuda:0' if disp == '0' else 'cpu')))
    for nombre, disp, fn, red, dev in modelos:
        r = res.setdefault(nombre, {})
        if disp == '0':
            m = evaluar(nombre, fn, imgs, nc)
            r.update({k: (v.tolist() if hasattr(v, 'tolist') else float(v)) for k, v in m.items()})
            # Parámetros de la red entrenada (antes de fusionar BatchNorm y con las dos cabezas)
            r['parametros_M'] = (contar_parametros(FuegoNet(red.nc, red.escala)) if isinstance(red,
     FuegoNet)
                                 else sum(p.numel() for p in red.parameters())) / 1e6
            try:
                r['GFLOPs'] = gflops(red.to(dev).float().eval(), dev)
            except Exception as ex:          # algunas capas de ultralytics no se dejan contar
                print('  GFLOPs no disponibles:', ex)
            r['ms_gpu'] = latencia(fn, muestra)
        else:
            torch.set_num_threads(a.hilos)
            r['ms_cpu'] = latencia(fn, muestra)
        print(f"  {nombre} {disp}: {r.get('ms_gpu' if disp == '0' else 'ms_cpu'):.1f} ms")
    with open(a.salida, 'w') as f:
        json.dump(res, f, indent=2)
    print('Guardado', a.salida)


if __name__ == '__main__':
    p = argparse.ArgumentParser()
    p.add_argument('--datos', required=True)
    p.add_argument('--fuegonet', default='')
    p.add_argument('--yolo', nargs='*', default=[])
    p.add_argument('--salida', default='evaluacion.json')
    p.add_argument('--hilos', type=int, default=4)
    principal(p.parse_args())
```

A.11. inferir.py

```
#!/usr/bin/env python3
"""Usar FuegoNet entrenada: cargar los pesos y detectar en una imagen.

    ~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.inferir mejor.pt foto.jpg [salida.jpg]

La clase Detector la usa también el nodo de ROS 2 (detector_yolo con un modelo
*.fuegonet.pt). Devuelve cajas en píxeles de la imagen ORIGINAL: deshace el
letterbox (escala r y relleno dx, dy).

Para inferir más rápido (como hace ultralytics):
  * cada BatchNorm se pliega en su convolución (fusionar), y
  * opcional (rect=True): la imagen se rellena solo hasta un múltiplo de 32. Es
    más rápido, pero FuegoNet pierde 0.013 de mAP50-95 (se entrenó con imágenes
    cuadradas); por omisión se usa el cuadrado de 640 x 640.
"""
import sys
import time
```

```
import cv2
import numpy as np
import torch

from .bloques import Conv
from .datos import letterbox
from .modelo import FuegoNet


@torch.no_grad()
def fusionar(modelo):
    """Pliega BatchNorm en la convolución: y = g (W x - mu) / sigma + b = W' x + b'."""
    for m in modelo.modules():
        if isinstance(m, Conv) and isinstance(m.bn, torch.nn.BatchNorm2d):
            bn, conv = m.bn, m.conv
            k = bn.weight / torch.sqrt(bn.running_var + bn.eps)
            nueva = torch.nn.Conv2d(conv.in_channels, conv.out_channels, conv.kernel_size, conv.
    stride,
                                    conv.padding, conv.dilation, conv.groups, bias=True).to(conv.
    weight.device)
            nueva.weight.copy_(conv.weight * k.view(-1, 1, 1, 1))
            nueva.bias.copy_(bn.bias - bn.running_mean * k)
            m.conv, m.bn = nueva, torch.nn.Identity()
    return modelo


class Detector:

    def __init__(self, pesos, dispositivo='0', rect=False):
        self.rect = rect
        self.disp = torch.device(f'cuda:{dispositivo}' if dispositivo.isdigit() and torch.cuda.
    is_available()
                                 else 'cpu')
        p = torch.load(pesos, map_location=self.disp)
        self.nombres = {int(k): v for k, v in p['nombres'].items()}
        self.imgsz = p.get('imgsz', 640)
        self.modelo = FuegoNet(p['nc'], p['escala']).to(self.disp).eval()
        self.modelo.load_state_dict(p['modelo'])
        fusionar(self.modelo)

    @torch.no_grad()
    def __call__(self, img_bgr, conf=0.25):
        """Lista de (x1, y1, x2, y2, confianza, clase) para una imagen BGR."""
        x, r, (dx, dy) = letterbox(img_bgr, self.imgsz, rect=self.rect)
        x = torch.from_numpy(np.ascontiguousarray(x[..., ::-1].transpose(2, 0, 1))).to(self.disp)
        x = x[None].float() / 255
        y = self.modelo(x)[0].float().cpu().numpy()
        y = y[y[:, 4] >= conf]                          # sin NMS: solo un umbral
        y[:, [0, 2]] = (y[:, [0, 2]] - dx) / r
        y[:, [1, 3]] = (y[:, [1, 3]] - dy) / r
        h, w = img_bgr.shape[:2]
        y[:, [0, 2]] = y[:, [0, 2]].clip(0, w)
        y[:, [1, 3]] = y[:, [1, 3]].clip(0, h)
        return y


class _Cajas:
    def __init__(self, d):
        t = torch.from_numpy(d)
```

```
        self.xyxy, self.conf, self.cls = t[:, :4], t[:, 4], t[:, 5]


class _Resultado:
    def __init__(self, d):
        self.boxes = _Cajas(d)


class ComoYolo:
    """Envoltura con la interfaz de ultralytics que usa detector_yolo:
    .names y .predict(img_bgr, conf=..., device=..., verbose=...)[0].boxes."""

    def __init__(self, pesos, dispositivo='0'):
        self.det = Detector(pesos, str(dispositivo))
        self.names = self.det.nombres

    def predict(self, img, conf=0.25, device=None, verbose=False):
        return [_Resultado(self.det(np.ascontiguousarray(img), conf))]


def dibujar(img, dets, nombres):
    colores = {0: (0, 60, 255), 1: (255, 200, 0)}
    for x1, y1, x2, y2, c, k in dets:
        col = colores.get(int(k), (255, 255, 255))
        cv2.rectangle(img, (int(x1), int(y1)), (int(x2), int(y2)), col, 2)
        cv2.putText(img, f'{nombres[int(k)]} {c:.2f}', (int(x1), max(14, int(y1) - 5)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, col, 2)
    return img


if __name__ == '__main__':
    pesos, ruta = sys.argv[1], sys.argv[2]
    salida = sys.argv[3] if len(sys.argv) > 3 else 'deteccion.jpg'
    det = Detector(pesos)
    img = cv2.imread(ruta)
    det(img)                                          # calentar
    t0 = time.perf_counter()
    dets = det(img)
    print(f'{len(dets)} detecciones en {(time.perf_counter() - t0) * 1000:.1f} ms')
    for d in dets:
        print(f'  {det.nombres[int(d[5])]:6s} {d[4]:.2f}  caja ' + ' '.join(f'{v:.1f}' for v in d
    [:4]))
    cv2.imwrite(salida, dibujar(img, dets, det.nombres))
```

A.12. exportar.py

```
#!/usr/bin/env python3
"""Exporta FuegoNet a ONNX (para la computadora a bordo: Jetson, Raspberry Pi, etc.).

    ~/.venvs/yolo/bin/python -m cuadrirotor_sim.red_fuego.exportar mejor.pt fuegonet_n.onnx [foto.
    jpg]

Como la cabeza no necesita NMS, el grafo exportado termina en un solo tensor
(1, 300, 6) = x1, y1, x2, y2, confianza, clase sobre la imagen 640 x 640 con
letterbox. Con onnxruntime instalado se comprueba que ONNX y PyTorch coinciden
sobre una foto (si se da) o una imagen al azar.
```

```
"""
import sys

import numpy as np
import torch

from .datos import letterbox
from .modelo import FuegoNet


def exportar(pesos, salida, foto=None):
    p = torch.load(pesos, map_location='cpu')
    m = FuegoNet(p['nc'], p['escala']).eval()
    m.load_state_dict(p['modelo'])
    x = torch.rand(1, 3, p.get('imgsz', 640), p.get('imgsz', 640))
    torch.onnx.export(m, (x,), salida, input_names=['imagen'], output_names=['detecciones'],
                      opset_version=17, dynamo=False)
    print('Exportado', salida)
    try:
        import onnxruntime as ort
    except ImportError:
        print('(onnxruntime no está instalado: no se comprueba)')
        return
    if foto:
        import cv2
        img = letterbox(cv2.imread(foto), x.shape[-1])[0][..., ::-1].transpose(2, 0, 1)
        x = torch.from_numpy(np.ascontiguousarray(img))[None].float() / 255
    s = ort.InferenceSession(salida, providers=['CPUExecutionProvider'])
    y_onnx = s.run(None, {'imagen': x.numpy()})[0]
    with torch.no_grad():
        y = m(x).numpy()
    # Las puntuaciones casi empatadas (cerca de cero) pueden salir en otro orden del
    # topk; se comparan solo las detecciones con confianza >= 0.25.
    k = int((y[0, :, 4] >= 0.25).sum())
    d = np.abs(y_onnx[0, :k, :5] - y[0, :k, :5]).max() if k else 0.0
    print(f'{k} detecciones con confianza >= 0.25; diferencia máxima ONNX vs PyTorch: {d:.2e}')


if __name__ == '__main__':
    exportar(sys.argv[1], sys.argv[2] if len(sys.argv) > 2 else 'fuegonet.onnx',
             sys.argv[3] if len(sys.argv) > 3 else None)
```